

UNIVERSIDADE DE SÃO PAULO

ESCOLA POLITÉCNICA

MATHEUS TAVARES DE ANDRADE

**PROJETO DE HARDWARE PARA VIOLAÇÃO DE BOOT
SEGURO VIA MULTIPLEXAÇÃO DE BIOS: UM ESTUDO DE
CASO COM ATM**

SÃO PAULO

2025

MATHEUS TAVARES DE ANDRADE

**PROJETO DE HARDWARE PARA VIOLAÇÃO DE BOOT
SEGURO VIA MULTIPLEXAÇÃO DE BIOS: UM ESTUDO DE
CASO COM ATM**

Trabalho de conclusão de curso apresentado à Escola Politécnica da Universidade de São Paulo para obtenção do grau de bacharel em Engenharia de Computação.

Área de Concentração:

Engenharia de Computação

Orientador:

Prof. Dr. Marcos Antonio Simplicio Junior

SÃO PAULO

2025

Autorizo a reprodução e divulgação total ou parcial deste trabalho, por qualquer meio convencional ou eletrônico, para fins de estudo e pesquisa, desde que citada a fonte.

Catálogo-na-publicação

Andrade, Matheus

Projeto de Hardware para Violação de Boot Seguro via Multiplexação de BIOS: Um Estudo de Caso com ATM / M. Andrade -- São Paulo, 2025.
77 p.

Trabalho de Formatura - Escola Politécnica da Universidade de São Paulo. Departamento de Engenharia de Computação e Sistemas Digitais.

1.Segurança de Computadores 2.Sistemas Embarcados 3.Hardware
I.Universidade de São Paulo. Escola Politécnica. Departamento de Engenharia de Computação e Sistemas Digitais II.t.

AGRADECIMENTOS

À minha família, em especial aos meus pais, que, mesmo sem terem tido a oportunidade de estudar, fizeram inúmeros sacrifícios para que eu pudesse chegar até aqui. Este trabalho só foi possível graças ao esforço, ao amor e às renúncias que dedicaram à minha formação.

À minha querida “profe”, por estar sempre ao meu lado, oferecer apoio nos momentos mais difíceis e compartilhar comigo cada desafio e cada conquista.

Aos meus amigos do LARC, que sempre acreditaram e continuam acreditando em mim, especialmente nos momentos em que eu mesmo duvido da minha capacidade. A amizade e o apoio de vocês foram fundamentais ao longo de toda esta trajetória.

Ao meu orientador e aos professores do projeto, por me introduzir ao mundo acadêmico e à pesquisa na área de segurança e de embarcados, guiando meus primeiros passos e contribuindo de maneira decisiva para o meu desenvolvimento profissional e intelectual.

A todos vocês, expresso meu mais profundo e sincero agradecimento.

“Only do what your heart tells you.”

Diana Spencer

ABSTRACT

This work investigates the possibility of violating the secure boot process of an automated teller machine ATM belonging to a financial institution in the midst of transitioning from legacy BIOS to UEFI. Considering the increasing sophistication of attacks that exploit vulnerabilities in embedded systems, the architecture of the device was analyzed with emphasis on the circuit responsible for firmware storage and its behavior during system initialization. The study focused on extracting and reverse engineering the BIOS image, physically mapping critical components, and assessing the feasibility of firmware modification through flash-memory multiplexing. The results show that the absence of continuous integrity-verification mechanisms allows manipulated images to be introduced during transitional stages of the boot process, thereby compromising the system's chain of trust. The findings indicate that the adopted security model does not account for attack vectors based on non-destructive physical interventions, highlighting the need for additional protective mechanisms in this type of platform.

Keywords: Computer Security; Embedded Systems; Hardware.

RESUMO

Este trabalho investiga a possibilidade de violação do processo de boot seguro de um caixa eletrônico pertencente a uma instituição bancária que se encontra em transição entre BIOS e UEFI. Considerando o aumento da sofisticação de ataques que exploram vulnerabilidades em sistemas embarcados, analisou-se a arquitetura do equipamento, com foco no circuito responsável pelo armazenamento do *firmware* e em seu comportamento durante a inicialização. O estudo concentrou-se na extração e engenharia reversa da imagem da BIOS, no mapeamento físico dos componentes críticos e na avaliação da viabilidade de adulteração do *firmware* por meio da multiplexação da memória *flash*. Os resultados evidenciam que a ausência de mecanismos de verificação contínua da integridade da memória permite a introdução de imagens manipuladas durante a transição entre etapas do *boot*, comprometendo a cadeia de confiança do sistema. O trabalho demonstra, portanto, que o modelo de segurança adotado não contempla vetores de ataque baseados em intervenções físicas não destrutivas, ressaltando a necessidade de mecanismos adicionais de proteção nesse tipo de plataforma.

Palavras-chave: Segurança de Computadores; Sistemas Embarcados; *Hardware*.

LISTA DE FIGURAS

1	Sofisticação e Conhecimento do Atacante vs. Ataque	18
2	Quantidade de Saques por Canal Presencial entre 2010 e 2020	19
3	Quantidade de ATMs no Brasil entre 2006 e 2022	20
4	Placa de desenvolvimento Arduino Nano	26
5	Dispositivo CH341A	27
6	Fluxograma correspondente à atividade do painel LCD	31
7	Fluxograma correspondente às atividades dos monitores	32
8	Fluxograma correspondente à atividade da placa de circuito	33
9	Pontos de contato do circuito	34
10	Localização do componente U24 no circuito	34
11	Configurações possíveis da memória <i>flash</i> W25Q64	36
12	Mapeamento de continuidade dos terminais da memória <i>flash</i>	37
13	Esquema elétrico do circuito da memória <i>flash</i>	37
14	Localização dos componentes U22 e U25 no circuito	38
15	Diagrama de funcionamento dos multiplexadores	40
16	Mapeamento de continuidade dos terminais dos multiplexadores	40
17	Esquema elétrico do circuito dos multiplexadores	41
18	Localização do componente U26 no circuito	42
19	Testes de continuidade do componente U26	43
20	Circuito parcial do componente U26	43
21	Leitura de memória para extração do firmware	44
22	Cálculo normalizado da entropia do firmware obtido	45
23	Retorno da execução do comando “binwalk BIOS.rom”	46
24	Diagrama de árvore contendo os arquivos extraídos pelo Binwalk	47
25	Imagens obtidas através da ferramenta Binwalk (Censuradas)	47
26	Principais seções do arquivo BIOS.rom	48
27	Informações presentes na região “Descriptor”	49
28	<i>Driver</i> “SeguroDxe” localizado na região BIOS	50
29	Texto legível presente no binário do arquivo “SeguroDxe.efi”	51
30	Aplicação “EdkShell” localizada na região BIOS	51
31	Exemplo de vulnerabilidade de condição de corrida	53
32	Fluxograma correspondente ao funcionamento observado do caixa	54
33	Exploração de condição de corrida	54
34	Monitoramento de sinais de componentes no osciloscópio	55
35	Trecho do Arquivo Binário Escolhido para Alteração	58

36	Configuração utilizada para <i>boot</i> via <i>protoboard</i>	59
37	Configuração utilizada para <i>boot</i> via inserção direta no barramento	60
38	Configuração utilizada para <i>boot</i> via inserção por fios no barramento	61
39	Fotos sob microscópio das modificações realizadas no circuito	61
40	Novo arranjo de contatos para a memória <i>flash</i>	62
41	Protótipo inicial para teste de <i>boot</i> pelos pinos externos	63
42	Esquema do circuito de multiplexação	64
43	Fios para conexão com o comutador do protótipo	65
44	Protótipo final para teste de <i>boot</i> pelos pinos externos	65
45	Shield conectado à placa-mãe	66
46	Distância estimada entre os pinos	73
47	Componente U26 soldado em adaptador SMD visto por meio de microscópio .	74
48	Circuito para Testes do Componente U26	75
49	Código de testes a ser embarcado no Arduino Nano	76
50	Comportamento do circuito para diferentes configurações de entrada	77

LISTA DE TABELAS

2	Descrição dos pinos do dispositivo	35
3	Descrição dos pinos do MUX1574PWR	38
4	Descrição dos pinos do 74LVC1G08	42
5	Possíveis vetores de exploração identificados, com impacto e dificuldade técnica associados	55

LISTA DE SIGLAS

AI	<i>Artificial Intelligence</i> (Inteligência Artificial)
ATM	<i>Automated Teller Machine</i> (Máquina de Autoatendimento)
BIOS	<i>Basic Input-Output System</i> (Sistema Básico de Entrada e Saída)
CI	Circuito Integrado
COFF	<i>Common Object File Format</i> (Formato Comum de Arquivo de Objeto)
CPU	<i>Central Processing Unit</i> (Unidade Central de Processamento)
DXE	<i>Driver eXecution Environment</i> (Ambiente de Execução de Drivers)
EDA	<i>Electronic Design Automation</i> (Automação de Projeto Eletrônico)
EDK	<i>EFI Development Kit</i> (Kit de Desenvolvimento EFI)
FAT	<i>File Allocation Table</i> (Tabela de Alocação de Arquivos)
FPGA	<i>Field Programmable Gate Array</i> (Matriz de Portas Programáveis em Campo)
IDE	<i>Integrated Development Environment</i> (Ambiente Integrado de Desenvolvimento)
LCD	<i>Liquid Crystal Display</i> (Tela de Cristal Líquido)
LED	<i>Light-Emitting Diode</i> (Diodo Emissor de Luz)
LZMA	<i>Lempel–Ziv–Markov Chain Algorithm</i> (Algoritmo de Cadeia de Lempel–Ziv–Markov)
NDA	<i>Non Disclosure Agreement</i> (Acordo de Confidencialidade)
PCI	Placa de Circuito Impresso
PDR	<i>Platform Data Region</i> (Região de Dados da Plataforma)
PE	<i>Portable Executable</i> (Executável Portátil)
PEI	<i>Pre-EFI Initialization</i> (Inicialização Pré-EFI)

PIN	<i>Personal Identification Number</i> (Número de Identificação Pessoal)
PTES	<i>Penetration Testing Execution Standard</i> (Padrão de Execução de Testes de Penetração)
SPI	<i>Serial Peripheral Interface</i> (Interface Serial Periférica)
TOCTOU	<i>Time-of-check, Time-of-use</i> (Tempo-de-verificação, Tempo-de-uso)
UEFI	<i>Unified Extensible Firmware Interface</i> (Interface Unificada de Firmware Extensível)
USB	<i>Universal Serial Bus</i> (Barramento Serial Universal)

SUMÁRIO

1	INTRODUÇÃO	16
1.1	MOTIVAÇÃO	16
1.1.1	Classificação dos Principais Tipos de Ataques	16
1.1.1.1	Ataques por Meios Físicos	16
1.1.1.2	Ataques por Meios Lógicos	17
1.1.1.3	Ataques por Observação e Inferência	17
1.1.2	Desafios Atuais e Tendências	17
1.1.3	Lacunas na literatura	18
1.2	OBJETIVOS	18
1.3	JUSTIFICATIVA	19
1.4	ORGANIZAÇÃO DO TRABALHO	20
2	PROCEDIMENTO METODOLÓGICO	22
2.1	INTERAÇÕES PRÉVIAS AO ENGAJAMENTO	22
2.2	COLETA DE INTELIGÊNCIA	22
2.3	MODELAGEM DE AMEAÇAS	22
2.4	ANÁLISE DE VULNERABILIDADES	23
2.5	EXPLORAÇÃO	23
2.6	PÓS-EXPLORAÇÃO	23
3	INTERAÇÕES PRÉVIAS AO ENGAJAMENTO	24
3.1	DO ESCOPO DE EXPLORAÇÃO	24
3.2	DA DIVULGAÇÃO PÚBLICA	24
4	COLETA DE INTELIGÊNCIA	26
4.1	DISPOSITIVOS DE	26
4.1.1	Arduino Nano	26
4.1.2	Programador CH341A	27
4.2	DISPOSITIVOS DE	28
4.2.1	Altium Designer	28
4.2.2	Arduino IDE	28
4.2.3	Flashrom	28
4.2.4	Binwalk	29
4.2.5	UEFITool	29

5	MODELAGEM DE AMEAÇAS	30
5.1	ANÁLISE DO	30
5.1.1	Análise Externa	30
5.1.1.1	Painel LCD	30
5.1.1.2	Monitores	31
5.1.1.3	Placa de Circuito	33
5.1.2	Análise Interna	33
5.1.2.1	Circuito Integrado 24 (U24)	34
5.1.2.1.1	<i>Identificação do Componente</i>	<i>34</i>
5.1.2.1.2	<i>Descrição dos Pinos</i>	<i>35</i>
5.1.2.1.3	<i>Funcionamento do Componente</i>	<i>36</i>
5.1.2.1.4	<i>Mapeamento dos Terminais</i>	<i>36</i>
5.1.2.1.5	<i>Circuito Parcial</i>	<i>37</i>
5.1.2.2	Circuitos Integrados 22 e 25 (U22 e U25)	38
5.1.2.2.1	<i>Identificação do Componente</i>	<i>38</i>
5.1.2.2.2	<i>Descrição dos Pinos</i>	<i>38</i>
5.1.2.2.3	<i>Funcionamento do Componente</i>	<i>39</i>
5.1.2.2.4	<i>Mapeamento dos Terminais</i>	<i>40</i>
5.1.2.2.5	<i>Circuito Parcial</i>	<i>40</i>
5.1.2.3	Circuito Integrado 26 (U26)	41
5.1.2.3.1	<i>Identificação do Componente</i>	<i>41</i>
5.1.2.3.2	<i>Descrição dos Pinos</i>	<i>42</i>
5.1.2.3.3	<i>Mapeamento dos Terminais</i>	<i>42</i>
5.1.2.3.4	<i>Circuito Parcial</i>	<i>43</i>
5.2	ANÁLISE DO FIRMWARE	44
5.2.1	Extração do Firmware da BIOS	44
5.2.2	Análise de Entropia	45
5.2.3	Investigação com Binwalk	45
5.2.4	Investigação com UEFITool	48
5.2.4.1	Análise da Estrutura da Imagem	48
5.2.4.1.1	<i>Região Descriptor</i>	<i>48</i>
5.2.4.1.2	<i>Região DevExp1</i>	<i>49</i>
5.2.4.1.3	<i>Região PDR</i>	<i>50</i>
5.2.4.1.4	<i>Região BIOS</i>	<i>50</i>
5.2.4.2	Busca de Módulos Proprietários	50
6	ANÁLISE DE VULNERABILIDADES	52
7	EXPLORAÇÃO	58
7.1	ETAPA 01 - ALTERAÇÃO DE <i>BYTES</i> DE TEXTO NO FIRMWARE	58

7.2	ETAPA 02 - INSERÇÃO DE MEMÓRIA EXTERNA NO BARRAMENTO .	59
7.2.1	Uso de <i>Protoboard</i> e <i>Jumpers</i>	59
7.2.2	Soldagem de Memória Sobre a Original	60
7.2.3	Verificação de Distâncias	60
7.3	ETAPA 03 - CONSTRUÇÃO DE TRILHAS NA PLACA	61
7.4	ETAPA 04 - <i>BOOT</i> PELOS PINOS EXTERNOS	62
7.5	ETAPA 05 - CIRCUITO DE CHAVEAMENTO	63
7.6	ETAPA 06 - MONTAGEM DO PROTÓTIPO	64
7.7	ETAPA 07 - EXECUÇÃO DO ATAQUE	66
8	PÓS-EXPLORAÇÃO	67
9	CONSIDERAÇÕES FINAIS	68
9.1	CONCLUSÕES DO PROJETO DE FORMATURA	68
9.2	CONTRIBUIÇÕES	68
9.3	PERSPECTIVAS DE CONTINUIDADE	69
	REFERÊNCIAS	70
	APÊNDICES	72
A	PROCEDIMENTOS PARA IDENTIFICAÇÃO - U26	73
A.1	IDENTIFICAÇÃO PARCIAL	73
A.2	REALIZAÇÃO DE TESTES	74
A.3	RESULTADOS	76

1 INTRODUÇÃO

Os ATMs, também conhecidos como caixas eletrônicos, constituem componentes essenciais da infraestrutura bancária contemporânea. Sua disponibilidade contínua e a interação direta com numerário físico tornam tais dispositivos alvos recorrentes de ataques físicos e lógicos. A evolução tecnológica desses terminais, paralelamente ao aumento da sofisticação das ações criminosas, resultou no surgimento de novas modalidades de fraude e exploração ([Tadeusz; Nowak, 2021](#)).

1.1 MOTIVAÇÃO

Inicialmente restritos a ações majoritariamente físicas, como vandalismo e arrombamento, os ataques passaram a explorar vulnerabilidades tecnológicas à medida que os ATMs foram progressivamente informatizados. Entre as primeiras técnicas amplamente sistematizadas destaca-se o *skimming*, baseado na captura clandestina dos dados do cartão e do PIN do usuário ([Sharma; Gupta, 2020](#)). Há também ataques lógicos, como instalação de *malware*, adulteração de firmware e técnicas de *jackpotting*, os quais ampliaram de forma significativa o impacto potencial dos incidentes bancários ([Tadeusz; Nowak, 2021](#); [Black Hat USA, 2010](#)).

1.1.1 Classificação dos Principais Tipos de Ataques

A literatura classifica os ataques, de modo geral, em duas categorias principais: físicos e lógicos ([Fraile et al., 2016](#)). Contudo, é possível incluir também uma categoria adicional que abrange ataques baseados em inferência.

1.1.1.1 Ataques por Meios Físicos

O *skimming* permanece uma das técnicas mais documentadas, consistindo na instalação de dispositivos falsos sobre o leitor de cartão, frequentemente acompanhados por câmeras ou teclados adulterados para capturar o PIN. Avanços recentes, como o *shim-*

ming, estendem essas capacidades por meio de microcircuitos inseridos entre o cartão e o leitor. Como contramedida, têm sido propostos sistemas de detecção de adulterações estruturais, conforme discutido em (Sharma; Gupta, 2020).

1.1.1.2 Ataques por Meios Lógicos

Os ataques lógicos exploram vulnerabilidades no ou na rede do terminal, frequentemente resultando em comprometimentos silenciosos e de elevado impacto. Dentre esses ataques, destacam-se a instalação de *malware* especializado, a exploração de interfaces de manutenção e a interceptação da comunicação entre ATM e servidor (Tadeusz; Nowak, 2021). Tais ataques são frequentemente empregados em operações coordenadas, comumente associadas ao crime organizado.

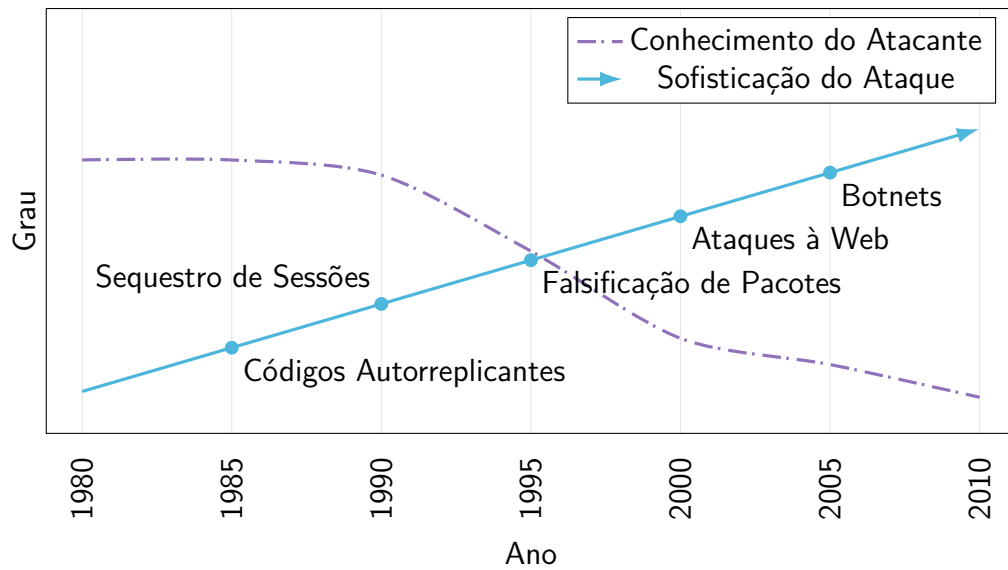
1.1.1.3 Ataques por Observação e Inferência

Os ataques baseados em observação direta têm ganhado destaque, especialmente com o emprego de técnicas de visão computacional. O estudo apresentado em (Shukla; Kumar; Singh, 2021) demonstra que métodos de aprendizado profundo são capazes de inferir PINs digitados em ATMs mesmo quando o usuário tenta cobrir o teclado com a mão, alcançando níveis expressivos de acurácia.

1.1.2 Desafios Atuais e Tendências

Apesar da adoção de tecnologias como sensores contra adulterações, biometria e mecanismos criptográficos, a literatura indica que a eficácia dessas medidas varia consideravelmente entre instituições e países. Estudos mostram que ataques continuam explorando falhas organizacionais, como ausência de padronização e insuficiência de inspeções periódicas (Sharma; Gupta, 2020). Ademais, novas modalidades de ataque baseadas em inteligência artificial tendem a intensificar os desafios impostos aos modelos atuais de defesa (Shukla; Kumar; Singh, 2021), agravadas pelo fato de que tais avanços têm reduzido significativamente o nível de conhecimento técnico necessário para a execução de ataques sofisticados, tal como mostra a Figura 1.

Figura 1 – Sofisticação e Conhecimento do Atacante vs. Ataque



Fonte: Adaptado de Ramirez (2017)

1.1.3 Lacunas na literatura

Os trabalhos existentes apontam a necessidade de estudos voltados ao contexto latino-americano, em que fatores socioeconômicos e características específicas da infraestrutura bancária influenciam o cenário de risco. Identifica-se, também, uma carência de bases públicas de incidentes e de abordagens que integrem segurança física, lógica e comportamental, o que evidencia oportunidades relevantes para pesquisas futuras (Tadeusz; Nowak, 2021).

1.2 OBJETIVOS

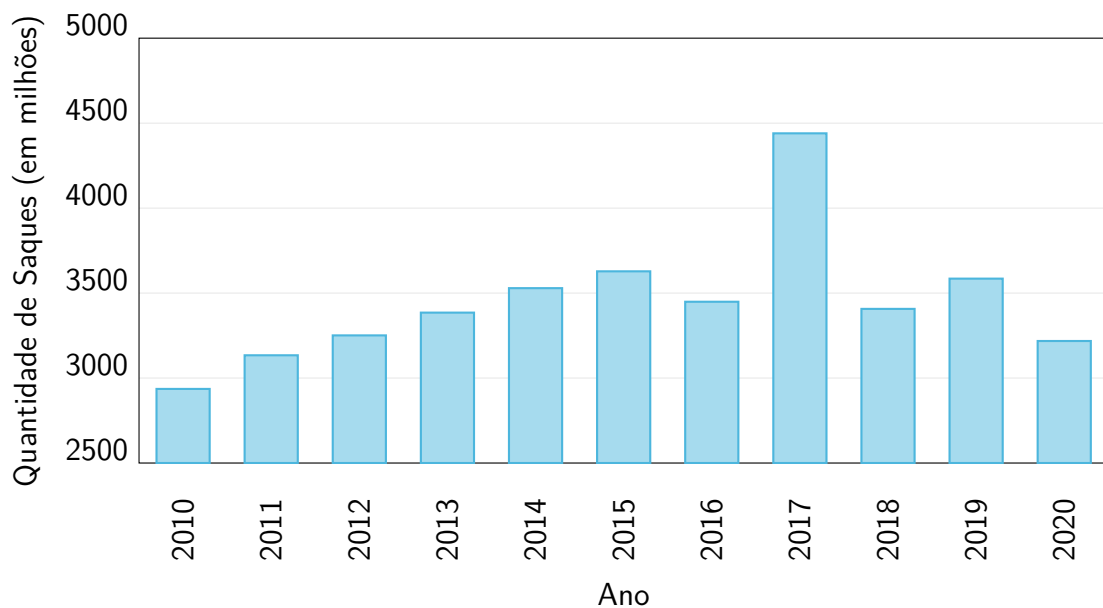
O objetivo geral deste trabalho é demonstrar a possibilidade de subversão da cadeia de segurança de um caixa eletrônico pertencente a uma instituição bancária que está em processo de migração da BIOS legada para UEFI. O problema de pesquisa decorre do fato de que as medidas de segurança atualmente adotadas concentram-se, em grande parte, na proteção contra ataques internos, baseando-se em verificações de integridade e mecanismos de validação de código incorporados ao processo de boot seguro. Entretanto, considera-se a possibilidade de exploração de vulnerabilidades presentes nas transições entre as etapas dessa cadeia, as quais não são contempladas pelos mecanismos internos de proteção. Ademais, diferentemente dos ataques anteriormente descritos, o cenário

analisado pressupõe o furto do circuito, o que potencializa vetores de exploração tanto físicos quanto lógicos.

1.3 JUSTIFICATIVA

Ainda que o uso de numerário tenha se reduzido em diversas economias desenvolvidas, pesquisas recentes apontam que países menos digitalizados e economias de renda média ainda dependem fortemente de transações em espécie. No Brasil, dados do Banco Central indicam que, embora a participação relativa dos ATMs na circulação de dinheiro tenha diminuído de maneira expressiva após 2017, os equipamentos continuam movimentando valores absolutos significativos ([Banco Central do Brasil, 2017a](#)), como mostra a Figura 2.

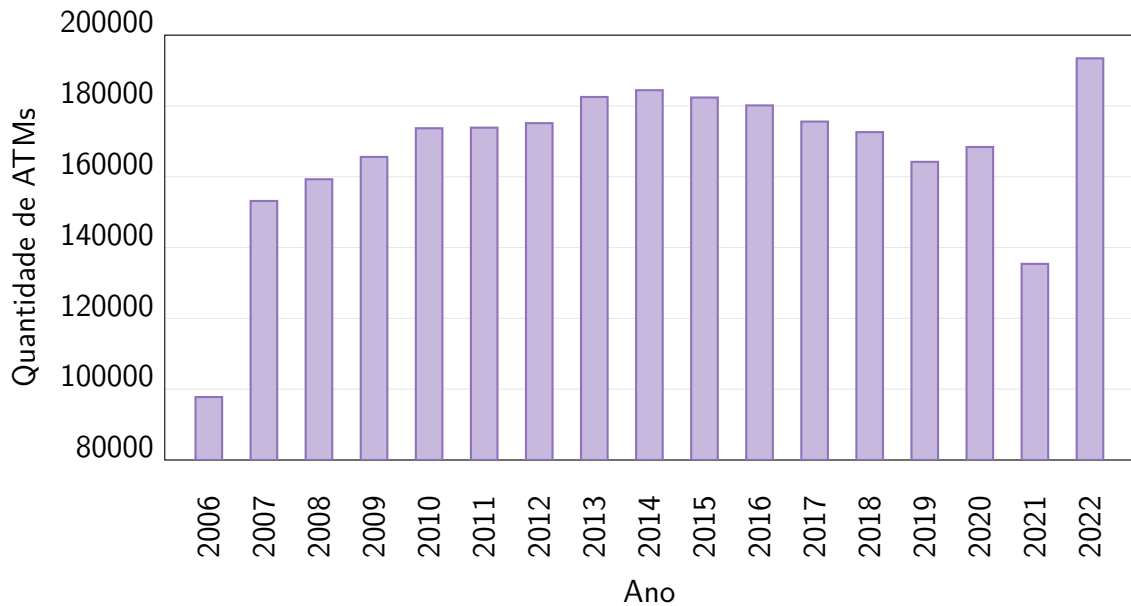
Figura 2 – Quantidade de Saques por Canal Presencial entre 2010 e 2020



Fonte: [Banco Central do Brasil \(2017b\)](#)

Este cenário reforça a necessidade de estudos que avaliem potenciais vulnerabilidades nos sistemas embarcados desses dispositivos e que proponham abordagens que contribuam para o aumento de sua segurança. Já a Figura 3 apresenta o número de terminais instalados no Brasil entre 2006 e 2022. Observa-se que, mesmo após a pandemia de COVID-19 e o período de *lockdown* entre 2020 e 2021, a quantidade de terminais alcançou o máximo de 193448 unidades no ano seguinte, contrariando a expectativa de redução em razão da migração para plataformas de pagamento *online* e da ampliação do uso do Pix ([Banco Central do Brasil, 2022](#)).

Figura 3 – Quantidade de ATMs no Brasil entre 2006 e 2022



Fonte: [Banco Central do Brasil \(2017a\)](#)

Por fim, a BIOS constitui o primeiro estágio de inicialização de um sistema embarcado e estabelece as bases de confiança para todos os demais componentes do sistema. Qualquer comprometimento nesse nível pode permitir que um invasor execute código malicioso antes mesmo da inicialização do sistema operacional, possibilitando ações como manipulação de periféricos e ocultação persistente da atividade ilícita.

1.4 ORGANIZAÇÃO DO TRABALHO

Este trabalho está estruturado em nove capítulos, organizados de modo a conduzir o leitor de forma progressiva desde o contexto do problema até a análise dos resultados obtidos.

O Capítulo 1 apresenta o tema investigado, discute a motivação, explicita os objetivos e a justificativa do estudo, além de situar o leitor no panorama atual de ataques a caixas eletrônicos.

O Capítulo 2 descreve o procedimento metodológico adotado, fundamentado em um arcabouço consolidado para testes de intrusão, detalhando as etapas que orientam todo o desenvolvimento da pesquisa.

O Capítulo 3 expõe as interações prévias ao engajamento, especificando os limites,

restrições e autorizações definidas pela instituição responsável pelo equipamento analisado.

O Capítulo 4 trata da coleta de inteligência, apresentando os dispositivos de e utilizados, suas funções e o papel desempenhado por cada um na condução das análises subsequentes.

O Capítulo 5 dedica-se à modelagem de ameaças, abordando a superfície de ataque do dispositivo, a análise de hardware, o mapeamento dos componentes críticos e a inspeção inicial do firmware.

O Capítulo 6 discute as vulnerabilidades identificadas, destacando seu impacto, sua relevância e seu potencial de exploração dentro do contexto estudado.

O Capítulo 7 relata a fase de exploração, descrevendo em detalhe os experimentos realizados para avaliar a viabilidade de comprometer o processo de *boot* seguro por meio da multiplexação da memória *flash*.

O Capítulo 8 aborda a etapa de pós-exploração, examinando as implicações práticas do comprometimento obtido e os possíveis desdobramentos para a segurança do dispositivo.

Por fim, o Capítulo 9 apresenta as considerações finais, reunindo as conclusões do estudo, suas contribuições e potenciais direções para continuidade desta linha de pesquisa.

2 PROCEDIMENTO METODOLÓGICO

A metodologia adotada baseia-se em um *framework* amplamente consolidado na literatura para a condução dos testes de intrusão, chamado de PTES ([The Penetration Testing Execution Standard \(2014\)](#)). O *framework* foi adaptado de modo a contemplar um cenário de ataque no qual o agente malicioso não precisa possuir conhecimento prévio acerca de aspectos como acessibilidade física, monitoramento e medidas de segurança do ambiente em que o objeto de estudo está inserido. Cada uma das etapas encontra-se nas seções a seguir.

2.1 INTERAÇÕES PRÉVIAS AO ENGAJAMENTO

Esta etapa envolve a firmação de um acordo entre as partes envolvidas no projeto no que diz respeito ao ambiente a ser considerado, ao escopo de ataque e às condições envolvidas no cenário.

2.2 COLETA DE INTELIGÊNCIA

O principal objetivo desta fase é fornecer subsídios para as etapas subsequentes do teste de intrusão, permitindo uma avaliação mais precisa dos vetores de ataque potencialmente exploráveis. No contexto deste projeto, essa etapa refere-se às tecnologias que podem ser empregadas para viabilizar tanto a análise quanto a execução dos ataques.

2.3 MODELAGEM DE AMEAÇAS

No contexto deste projeto, esta etapa abrange a identificação de ativos críticos (como dados sensíveis e sistemas essenciais), a análise das superfícies de ataque (pontos de entrada disponíveis ao atacante) e a avaliação do impacto e da probabilidade de ocorrência de cada ameaça.

2.4 ANÁLISE DE VULNERABILIDADES

Esta fase consiste na aplicação de técnicas automatizadas e manuais voltadas à identificação, validação e priorização das falhas de segurança existentes no ambiente-alvo. Seu propósito é revelar pontos de comprometimento que possam ser explorados por agentes maliciosos, servindo de base para a fase subsequente do teste.

2.5 EXPLORAÇÃO

Esta etapa é dedicada à exploração prática das vulnerabilidades previamente identificadas, com o objetivo de obter acesso não autorizado, realizar escalonamento de privilégios ou comprometer integralmente os sistemas envolvidos. Trata-se de uma fase essencial para verificar a viabilidade real de exploração das falhas e os riscos a elas associados.

2.6 PÓS-EXPLORAÇÃO

Nesta fase, avalia-se o impacto completo de uma intrusão bem-sucedida, mapeando o alcance potencial das ações de um atacante após a violação inicial. Tal etapa permite compreender a extensão dos possíveis danos e fornece subsídios para a proposição de medidas de contenção e mitigação adequadas.

3 INTERAÇÕES PRÉVIAS AO ENGAJAMENTO

No âmbito desta pesquisa, o objeto de estudo foi disponibilizado exclusivamente para fins de experimentação e análise, desconsiderando-se, portanto, a realização de invasões reais em ambientes empresariais ou qualquer interação com profissionais que atuam nesses locais. Eventuais dúvidas sobre o funcionamento de partes específicas do sistema podem ser esclarecidas com o apoio da parte solicitante. Ressalta-se que há um NDA vigente, além de autorização formal para a execução dos testes. Toda informação destinada à divulgação pública deverá ser previamente analisada e aprovada.

Os requisitos apresentados a seguir descrevem as exigências estabelecidas pela empresa, correspondentes às decisões de projeto acordadas entre as partes.

3.1 DO ESCOPO DE EXPLORAÇÃO

- 1) Está autorizada a exposição da placa de circuito a ataques invasivos, desde que não destrutivos;
- 2) **NÃO** está autorizada a manipulação do perímetro de segurança;
- 3) Está autorizada a extração de *firmwares* e a realização de engenharia reversa, desde que as cláusulas anteriores sejam respeitadas;
- 4) Em caso de danos acidentais ao equipamento, especialmente aqueles decorrentes dos ataques realizados, o ocorrido deve ser relatado. Não haverá necessidade de ressarcimento pelo dano.

3.2 DA DIVULGAÇÃO PÚBLICA

- 1) **NÃO** está autorizada a divulgação de vídeos envolvendo o equipamento;
- 2) Está autorizada a publicação de documentos acadêmicos, desde que não contenham menção ao nome da corporação ou que haja descaracterização do equipamento;

- 3) **NÃO** está autorizado o uso de AI para verificação de códigos ou binários;
- 4) **NÃO** está autorizado o uso de serviços de armazenamento em nuvem para códigos-fonte ou arquivos binários.

Em virtude dessas restrições, o trabalho aqui apresentado encontra-se descaracterizado e limitado, e qualquer divulgação explícita deve submeter-se a um embargo a ser definido pela corporação.

4 COLETA DE INTELIGÊNCIA

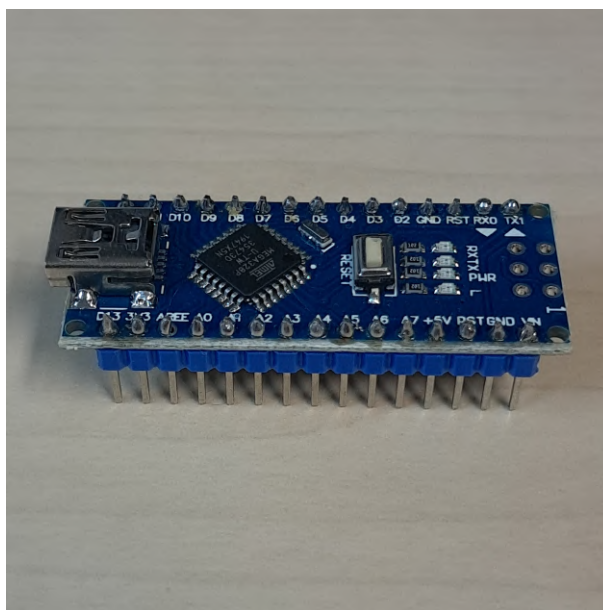
Neste capítulo são apresentados os principais aplicativos e dispositivos empregados no avanço do projeto. Ferramentas como editores de texto, imagens e outros utilitários não foram incluídas, pois tiveram caráter apenas auxiliar e não desempenharam papel central nas etapas de desenvolvimento.

4.1 DISPOSITIVOS DE

4.1.1 Arduino Nano

O Arduino Nano ([Arduino, 2025b](#)) é uma placa de desenvolvimento de arquitetura compacta, projetada para aplicações embarcadas que exigem baixo consumo de energia e dimensões reduzidas. Ele oferece uma solução funcionalmente equivalente a sua contraparte mais conhecida (Arduino Uno), porém em um formato significativamente menor, adequado para protótipos integrados diretamente a placas de circuito impresso ou sistemas eletrônicos com limitações de espaço.

Figura 4 – Placa de desenvolvimento Arduino Nano



Fonte: Autor

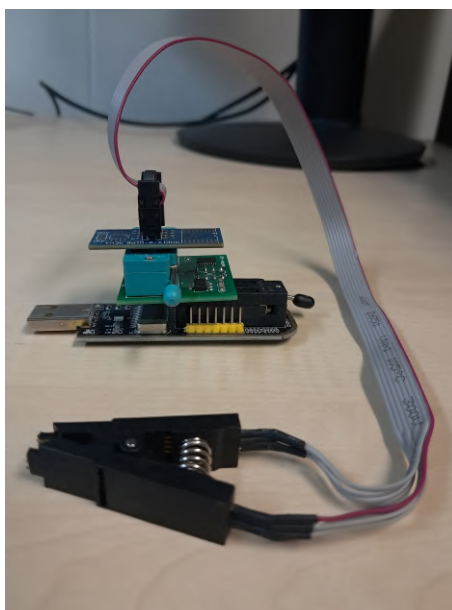
A placa disponibiliza um conjunto abrangente de pinos de entrada e saída digitais, permitindo o desenvolvimento de aplicações que variam desde automação simples até sistemas embarcados de maior complexidade. Uma de suas características distintivas é a capacidade de ser alimentado por fonte externa ou via USB, tornando a integração com computadores e outros dispositivos mais simples durante o processo de desenvolvimento.

O Arduino Nano utiliza um conversor USB–serial dedicado, permitindo a gravação do firmware e a comunicação com o ambiente Arduino IDE sem necessidade de um gravador externo. Além disso, sua compatibilidade com bibliotecas e ferramentas do ecossistema Arduino facilita a prototipagem rápida, reduzindo a complexidade do desenvolvimento de sistemas embarcados.

4.1.2 Programador CH341A

O CH341A ([Nanjing Qinheng Microelectronics, 2025](#)) é um pequeno programador USB usado para ler e gravar memórias EEPROM e *flash* SPI, como as encontradas em BIOS de computadores, roteadores e outros aparelhos eletrônicos. Ele permite acessar diretamente o conteúdo desses chips, sendo muito útil para regravar BIOS corrompidas, fazer backups de firmware ou restaurar dispositivos. Com o adequado, é possível editar, copiar ou substituir o conteúdo de memórias de forma prática.

Figura 5 – Dispositivo CH341A



Fonte: Autor

4.2 DISPOSITIVOS DE

4.2.1 Altium Designer

O Altium Designer ([Altium LLC, 2025](#)) é um profissional de projeto eletrônico assistido por computador (EDA), utilizado para o desenvolvimento de PCI, esquemáticos e *layouts* eletrônicos complexos. Amplamente adotado em engenharia eletrônica, automação e pesquisa em , a aplicação possibilita o projeto, a simulação e a documentação dos circuitos, integrando todas as etapas do processo de desenvolvimento, do conceito inicial ao protótipo.

4.2.2 Arduino IDE

O Arduino IDE ([Arduino, 2025a](#)) constitui o ambiente de desenvolvimento integrado oficial destinado à programação de plataformas baseadas na arquitetura Arduino. Ele fornece uma interface unificada para a criação, edição, compilação e transferência de códigos para os microcontroladores presentes nas placas. O ambiente incorpora um editor com realce de sintaxe, um compilador compatível com a linguagem C/C++ e uma ferramenta de envio do código compilado por meio de interface USB. Adicionalmente, disponibiliza um gerenciador de bibliotecas que facilita a integração de módulos externos e um monitor serial que permite realizar depuração e acompanhar, em tempo real, dados transmitidos pelo dispositivo. Em virtude de sua simplicidade, ampla compatibilidade e forte suporte comunitário, o Arduino IDE é amplamente empregado tanto em atividades de ensino quanto em experimentação e prototipagem no domínio da eletrônica embarcada.

4.2.3 Flashrom

O Flashrom ([Flashrom, 2025](#)) é uma ferramenta destinada à leitura, escrita, verificação e apagamento de memórias *flash* (ROM, BIOS, EEPROM, entre outras). É frequentemente empregada em coleta de informação de em baixo nível, recuperação de firmwares corrompidos e regravação de BIOS/UEFI.

4.2.4 Binwalk

O Binwalk ([ReFirm Labs, 2025](#)) é uma ferramenta voltada à análise e à engenharia reversa de firmwares, permitindo identificar e extrair arquivos e sistemas de arquivos incorporados em imagens binárias. Utilizado em pesquisas sobre segurança de dispositivos IoT, engenharia reversa e análise de vulnerabilidades, o Binwalk auxilia na decomposição de firmwares em seus componentes, o que facilita o estudo de sua estrutura interna.

4.2.5 UEFITool

O UEFITool ([LongSoft, 2025](#)) é um utilitário de código aberto empregado para analisar, modificar e extrair o conteúdo de firmwares UEFI (Unified Extensible Firmware Interface), que substituem as BIOS tradicionais em computadores modernos. É amplamente utilizado em pesquisas de segurança da informação, engenharia reversa de firmware e estudos sobre vulnerabilidades em sistemas embarcados. A ferramenta permite explorar a estrutura interna de imagens UEFI, possibilitando a inspeção de componentes binários e a identificação de eventuais falhas de segurança.

5 MODELAGEM DE AMEAÇAS

Este capítulo aborda a terceira etapa do procedimento metodológico, cujo objetivo é reunir o máximo de informações possíveis sobre o objeto de estudo. Nele são apresentados as informações obtidas para subsidiar a formulação de uma hipótese de viabilidade de ataque, que constitui o foco da etapa subsequente.

5.1 ANÁLISE DO

Esta seção descreve a etapa de análise de ameaças sob a perspectiva do circuito, conforme apresentado na Seção [2.3](#).

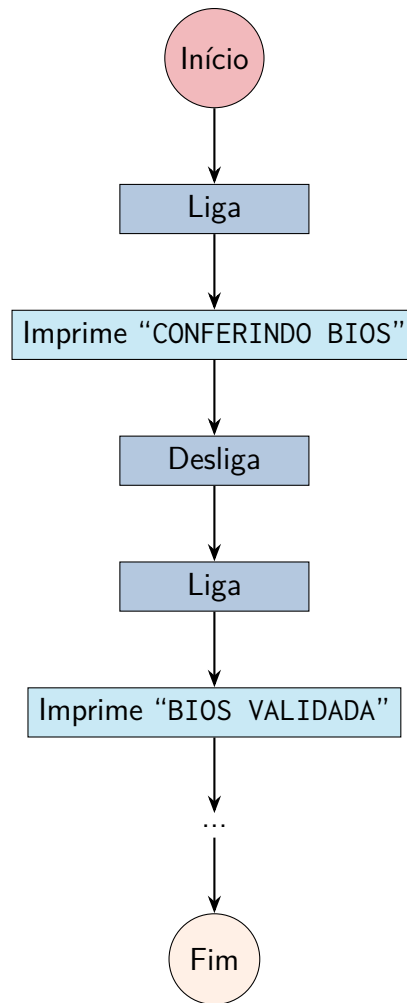
5.1.1 Análise Externa

Esta seção descreve o estudo do funcionamento do caixa eletrônico da perspectiva de caixa preta, ou seja, sem a manipulação direta com o circuito do equipamento, mas com a possibilidade de vê-lo em funcionamento. Além da observação da placa de circuito, estão considerados nessa parte o monitor principal, o monitor secundário e o painel LCD. Para facilitar o entendimento, os comportamentos serão descritos na forma de Diagramas.

5.1.1.1 Painel LCD

A Figura [6](#) apresenta, de forma simplificada, as ações executadas pelo painel LCD do caixa eletrônico. Após a validação da BIOS, são exibidas mensagens de verificação do bootloader, do kernel, do sistema operacional e da aplicação principal. Tais mensagens foram omitidas por não acrescentarem informações relevantes ao estudo.

Figura 6 – Fluxograma correspondente à atividade do painel LCD

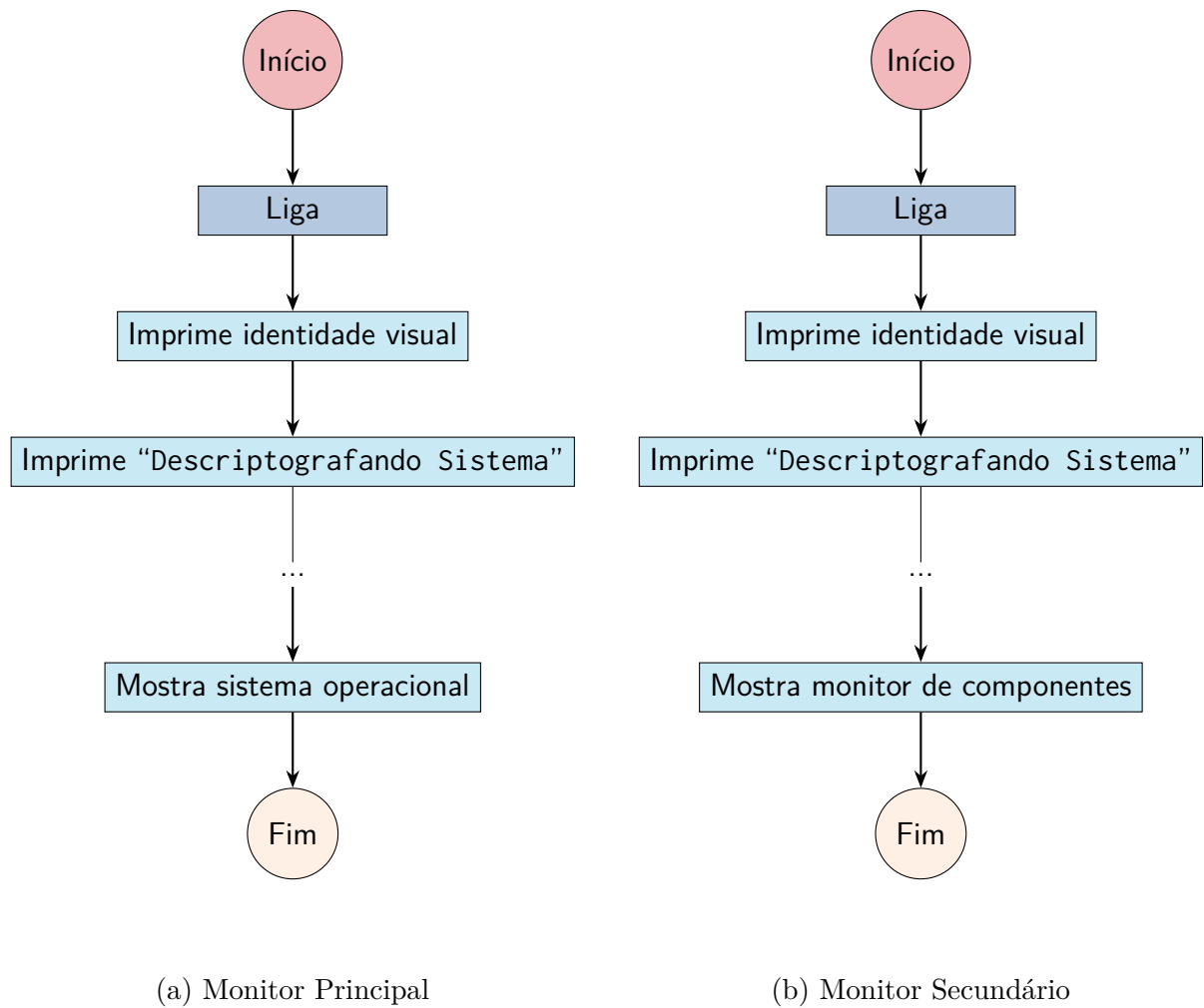


Fonte: Autor

5.1.1.2 Monitores

A Figura 7 apresenta, de forma simplificada, as ações executadas pelos monitores do sistema. Após a descritografia e a exibição de informações semelhantes às fornecidas pela aplicação “dmesg” do kernel, cada monitor passa a exibir um aplicativo distinto.

Figura 7 – Fluxograma correspondente às atividades dos monitores



Fonte: Autor

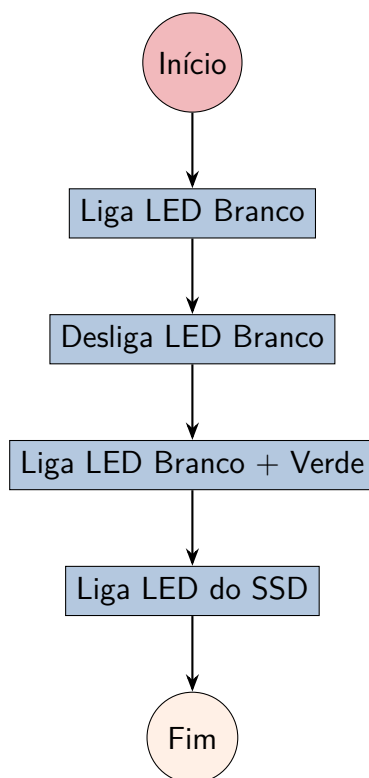
No monitor principal (Figura 7a), são exibidas as interfaces e aplicações com as quais o usuário pode interagir por meio do teclado do equipamento, permitindo realizar ações como saque, depósito e verificação de extrato, entre outras. No monitor secundário (Figura 7b), é exibido um que lista os periféricos conectados e apresenta o respectivo estado de cada um (“DESLIGADO” ou “LIGADO”).

O monitor secundário foi utilizado exclusivamente para acompanhar possíveis danos a componentes de durante o processo de pesquisa. Trata-se de um módulo removível e que não integra o escopo deste estudo.

5.1.1.3 Placa de Circuito

A Figura 8 apresenta, de forma simplificada, o comportamento observado na placa de circuito do caixa eletrônico. Como, nessa etapa, a análise foi exclusivamente visual, a atenção concentrou-se no comportamento dos LEDs.

Figura 8 – Fluxograma correspondente à atividade da placa de circuito



Fonte: Autor

O desligamento e a energização da placa correspondem ao mesmo evento representado no fluxograma do painel LCD, ou seja, ocorrem no mesmo intervalo de tempo.

5.1.2 Análise Interna

Esta seção descreve o estudo do funcionamento do circuito. São apresentados os principais componentes que o integram, bem como suas propriedades, de modo a fornecer a base necessária para as análises desenvolvidas nas seções seguintes. Para esta etapa, foi feito um mapeamento inicial de todos os pontos de contato do circuito, ou seja, regiões nas quais é possível interagir com um equipamento externo. A Figura 9 mostra os terminais obtidos para análise, representados por quadrados e retângulos de cor branca.

5.1.2.1.2 Descrição dos Pinos

A Tabela 2 apresenta, de forma sintetizada, a função de cada pino do circuito integrado e o respectivo sentido de operação.

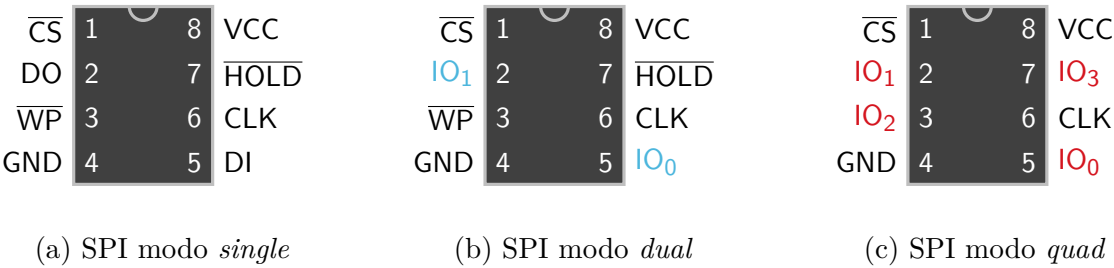
Tabela 2 – Descrição dos pinos do dispositivo

Pino	Tipo	Descrição
$\overline{CS}$	Entrada	Pino responsável pela ativação e desativação do chip, operando com lógica invertida. Em nível lógico 0, o dispositivo permanece ativo; em nível lógico 1, seus terminais entram em alta impedância (high-Z).
DO	Saída / Bidirecional	Pino destinado à saída de dados ou, dependendo do modo de operação (ver Seção 5.1.2.1.3), configurável como bidirecional.
$\overline{WP}$	Entrada / Bidirecional	Pino que controla a permissão de escrita, operando com lógica invertida. Em nível lógico 0, a escrita é permitida; em nível lógico 1, é desabilitada. Pode atuar como pino bidirecional de dados a depender do modo de operação (ver Seção 5.1.2.1.3).
GND	Alimentação	Pino de referência elétrica (terra) do circuito.
DI	Entrada / Bidirecional	Pino destinado à entrada de dados e comandos, podendo ser configurado como bidirecional de acordo com o modo de operação (ver Seção 5.1.2.1.3).
CLK	Entrada	Pino que recebe o sinal de <i>clock</i> da memória, suportando frequências de até 133 MHz.
$\overline{HI}$	Entrada / Bidirecional	Pino que permite interromper uma operação de escrita ou envio de comandos, ativo em nível baixo. Se $\overline{HI}$ e $\overline{CS}$ estiverem simultaneamente em nível lógico 0, os sinais de dados e <i>clock</i> são ignorados. Pode ser configurado como pino bidirecional a depender do modo de operação do chip (ver Seção 5.1.2.1.3).

5.1.2.1.3 Funcionamento do Componente

A Figura 11 apresenta as possíveis configurações de SPI do chip, bem como a taxa de dados correspondente que pode ser transferida em cada configuração. Os pinos destacados na cor vermelha indicam que o terminal em questão possui lógica invertida.

Figura 11 – Configurações possíveis da memória *flash* W25Q64



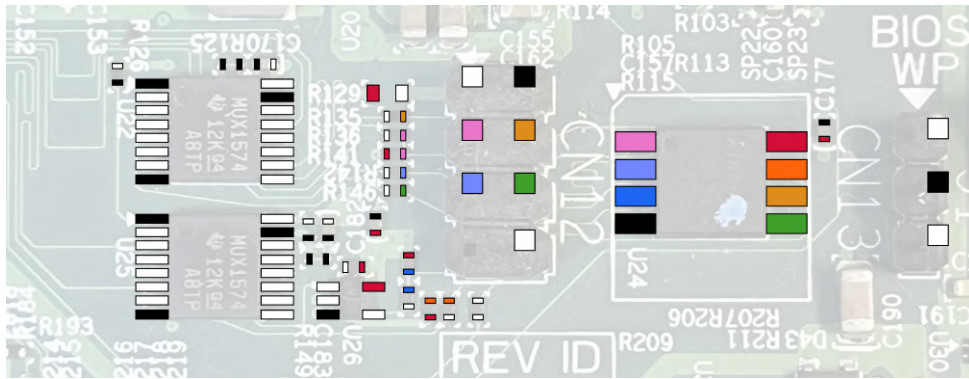
(Fonte: adaptado de [Winbond \(2025\)](#))

A relevância dessa informação está associada, sobretudo, ao controle de escrita e à velocidade de operação. No modo *quad*, não há proteção de escrita, uma vez que o terminal 3 passa a atuar como pino bidirecional de dados. Ademais, caso a memória opere nesse modo, a construção de um circuito externo torna-se dificultada devido às exigências de integridade de sinal em altas frequências, visto que a operação pode atingir valores equivalentes a quatro vezes o *clock* básico, alcançando aproximadamente 532 MHz. Esse cenário também compromete a análise e a medição dos sinais, dado que o osciloscópio e o analisador lógico disponíveis no laboratório possuem capacidades máximas de apenas 100 MHz e 50 MHz, respectivamente.

5.1.2.1.4 Mapeamento dos Terminais

O mapeamento do componente está indicado na Figura 12. Como pode ser visto, cinco dos oito terminais podem ser mapeados para o CN12, com o acréscimo do pino de alimentação, que está cortado pela ausência de um resistor na trilha.

Figura 12 – Mapeamento de continuidade dos terminais da memória *flash*

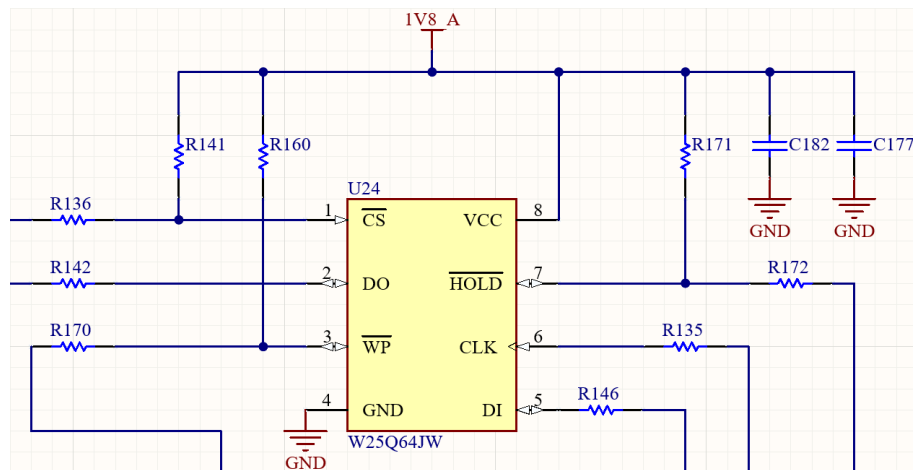


Fonte: Autor

5.1.2.1.5 Circuito Parcial

A Figura 13 mostra o esquema elétrico resultante do mapeamento. Observa-se que todo sinal passa por um resistor antes de ser acessado pelos multiplexadores.

Figura 13 – Esquema elétrico do circuito da memória *flash*



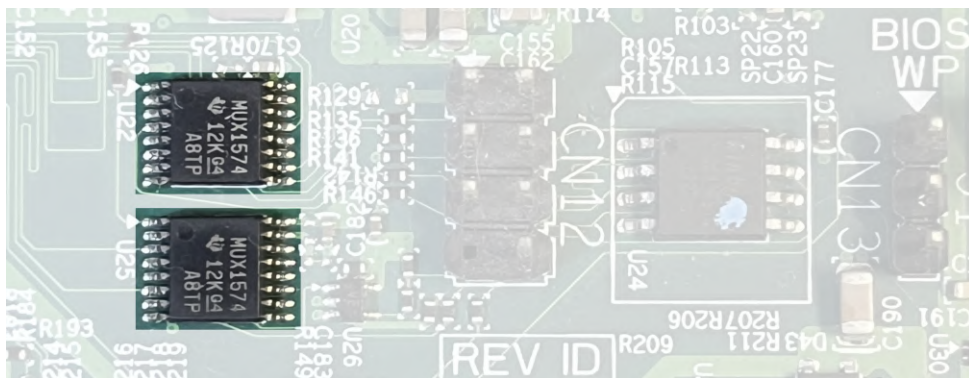
Fonte: Autor

5.1.2.2 Circuitos Integrados 22 e 25 (U22 e U25)

5.1.2.2.1 Identificação do Componente

O CI MUX1574PWR possui quatro chaveadores bidirecionais, funcionando tanto para sinais analógicos quanto digitais. Possui um intervalo de alimentação abrangente ($1.5V < VDD < 5.5V$) e compatibilidade lógica com sinais de 1.8V, o que é necessário no cenário deste projeto (Texas Instruments, 2018).

Figura 14 – Localização dos componentes U22 e U25 no circuito

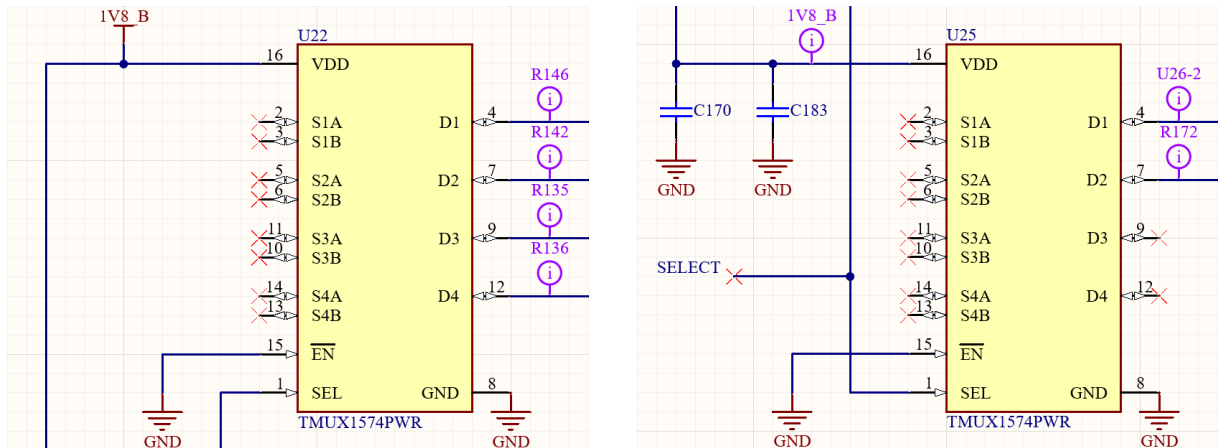


Pino	Tipo	Descrição
D1	Bidirecional	Pino de dreno 1. Pode atuar como entrada ou saída.
S2A	Bidirecional	Pino de fonte 2A. Pode operar como entrada ou saída.
S2B	Bidirecional	Pino de fonte 2B. Pode operar como entrada ou saída.
D2	Bidirecional	Pino de dreno 2. Pode operar como entrada ou saída.
GND	Alimentação	Pino de referência (terra, 0 V).
D3	Bidirecional	Pino de dreno 3. Pode atuar como entrada ou saída.
S3B	Bidirecional	Pino de fonte 3B. Pode operar como entrada ou saída.
S3A	Bidirecional	Pino de fonte 3A. Pode operar como entrada ou saída.
D4	Bidirecional	Pino de dreno 4. Pode atuar como entrada ou saída.
S4B	Bidirecional	Pino de fonte 4B. Pode operar como entrada ou saída.
S4A	Bidirecional	Pino de fonte 4A. Pode operar como entrada ou saída.
EN	Entrada	Pino de habilitação ativa em nível alto. Quando alto, todos os comutadores são desligados; quando baixo, o pino SEL controla a seleção dos caminhos.
VDD	Alimentação	Pino de alimentação positiva.

5.1.2.2.3 Funcionamento do Componente

A Figura 15 apresenta o circuito equivalente do circuito integrado. Os comutadores atuam como chaves analógicas, de modo que suas entradas e saídas operam de forma bidirecional.

Figura 17 – Esquema elétrico do circuito dos multiplexadores



(a) Circuito parcial do multiplexador U22

(b) Circuito parcial do multiplexador U25

Fonte: Autor

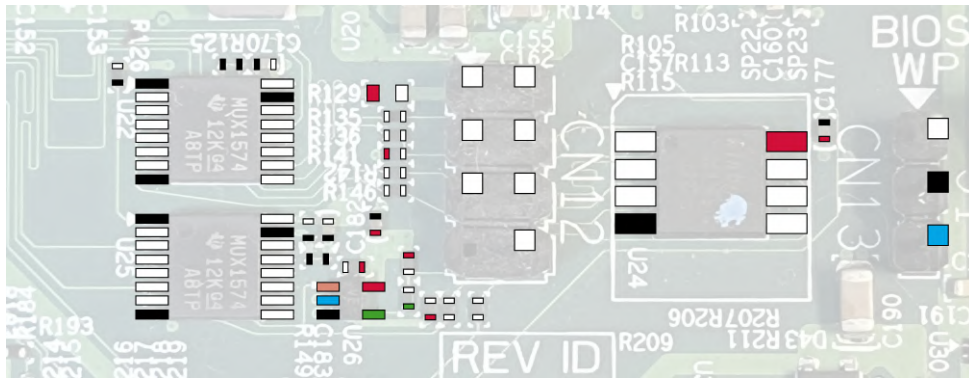
Os terminais D3, S3A, S3B, D4, S4A e S4B foram monitorados com um osciloscópio e com um multímetro desde a inicialização do equipamento até a execução e o encerramento da cadeia de *boot*. Não foram observadas atividades que indicassem entrada ou saída de sinais, tampouco a presença de grampeamento dos pinos em pontos de alimentação, tanto na região interna do circuito quanto nas áreas adjacentes. Desse modo, é possível afirmar que eles não estão em uso.

5.1.2.3 Circuito Integrado 26 (U26)

5.1.2.3.1 Identificação do Componente

Trata-se do componente identificado como 74LVC1G08, uma porta AND de duas entradas cuja faixa de alimentação permitida varia entre 1.65 V e 1.95 V. A Figura 18 apresenta sua posição no circuito de *boot*. Devido ao reduzido porte físico do dispositivo, sua identificação completa precisou ser realizada de forma experimental; as etapas que compõem esse processo encontram-se descritas no Apêndice A.1.

Figura 19 – Testes de continuidade do componente U26

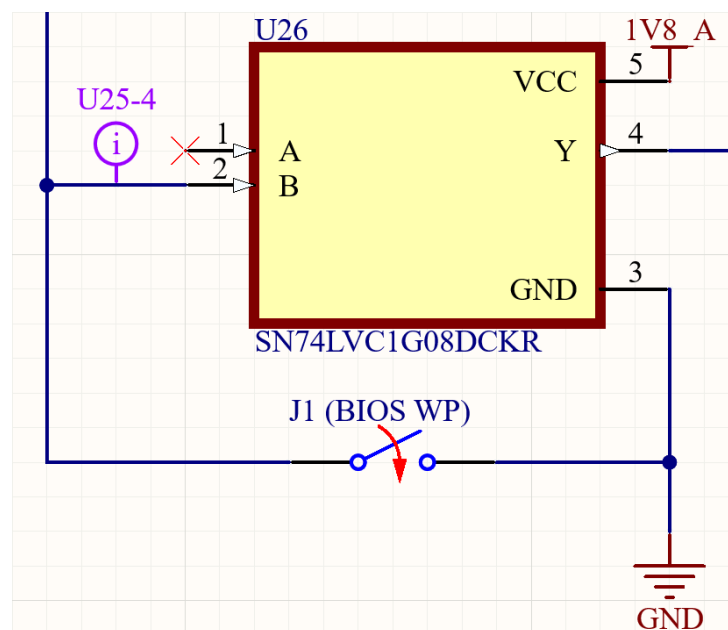


Fonte: Autor

5.1.2.3.4 Circuito Parcial

A Figura 20 apresenta o esquema elétrico da região do circuito responsável pelo chaveamento dos sinais da porta AND. Pode-se notar que o circuito integrado em questão exerce atividade no que diz respeito à proteção contra a escrita na memória *flash*, visto que há um caminho direto para o pino 3 desta, além de que há continuidade entre o terminal do meio do componente e o pino 3 do barramento J1 (BIOS WP).

Figura 20 – Circuito parcial do componente U26



Fonte: Autor

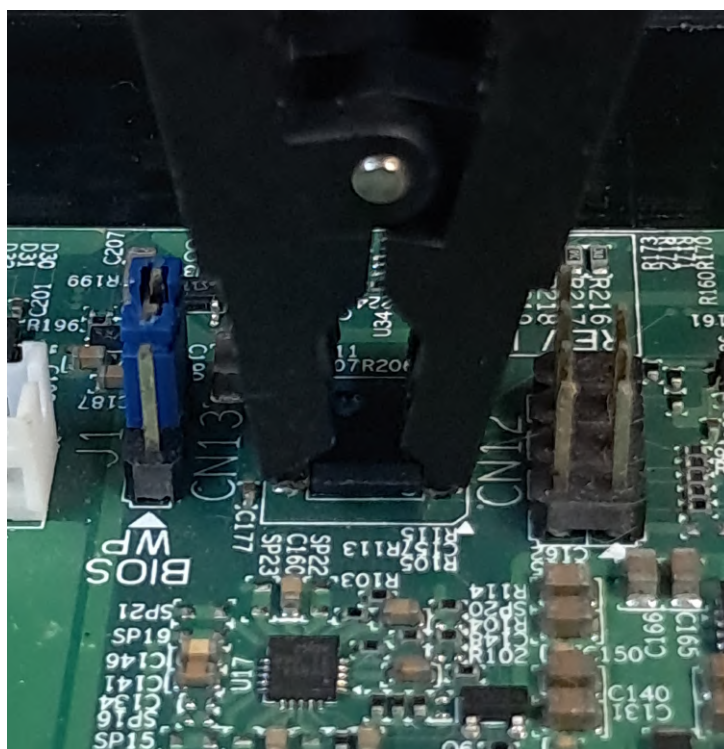
5.2 ANÁLISE DO FIRMWARE

Esta seção descreve as ações realizadas para obter informações sobre o firmware utilizado, abrangendo desde sua extração da memória *flash* até os procedimentos empregados em sua posterior análise e manipulação.

5.2.1 Extração do Firmware da BIOS

A primeira etapa consiste na extração do firmware da memória *flash*. Para isso, foram utilizados o programador CH341A, uma pinça de memória *flash* e o *flashrom*.

Figura 21 – Leitura de memória para extração do firmware



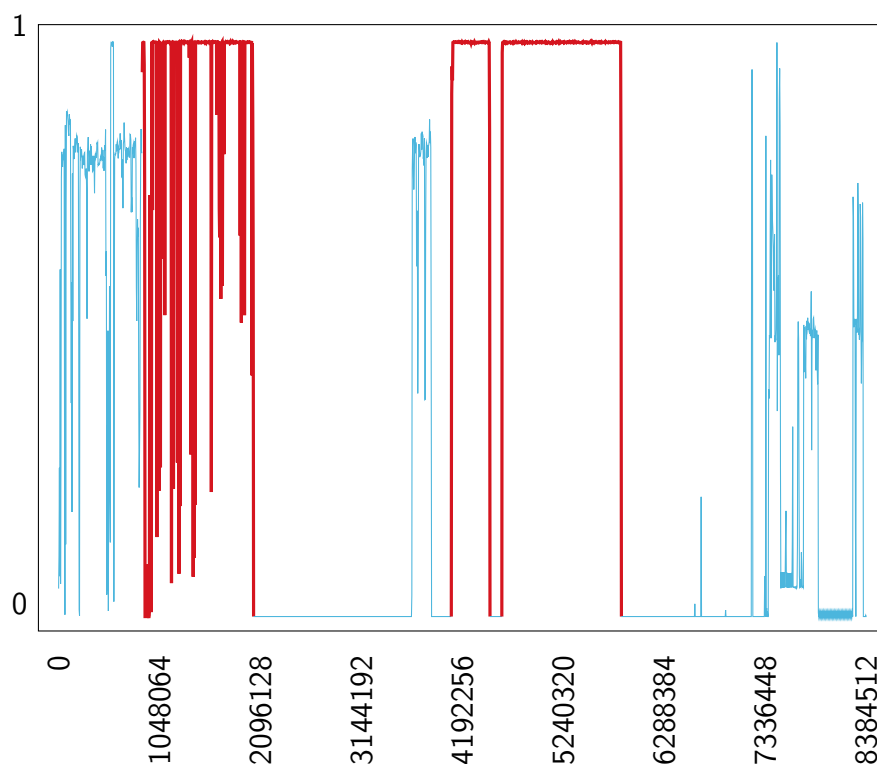
Fonte: Autor

A Figura 21 mostra como o procedimento é realizado. Ressalta-se que, nesta situação, os pinos da região “BIOS WP” não exercem a função de proteção contra escrita que deveriam, pois isso está sendo feito diretamente do chip.

5.2.2 Análise de Entropia

A Figura 22 apresenta o gráfico resultante do cálculo de entropia realizado sobre o arquivo binário extraído. Observa-se a presença de três regiões de entropia elevada: entre os endereços 868352 e 2019328, entre 4079616 e 4472832 e entre 4603904 e 5832704. Esse comportamento indica que tais regiões estão criptografadas ou comprimidas.

Figura 22 – Cálculo normalizado da entropia do firmware obtido



Fonte: Autor

Ressalta-se que a análise de entropia foi realizada inicialmente porque investigações baseadas apenas em *s* específicos podem produzir falsos positivos ou deixar de identificar elementos potencialmente relevantes.

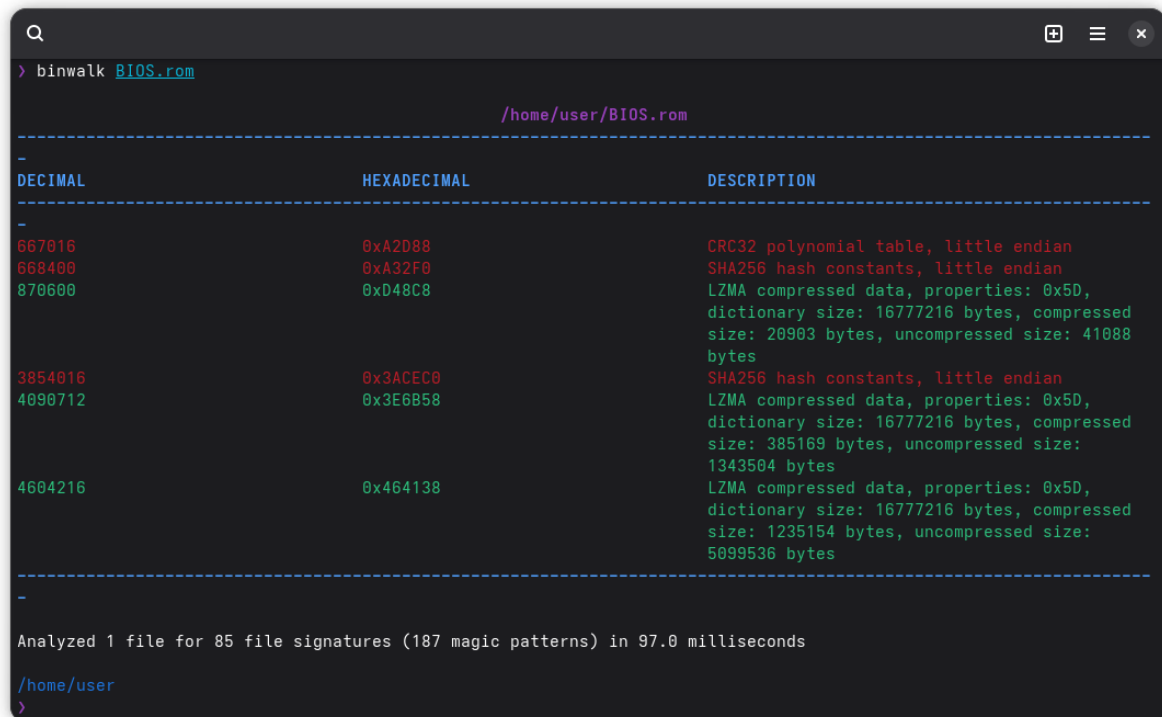
5.2.3 Investigação com Binwalk

Esta seção aborda o processo de obtenção de informações por meio da ferramenta Binwalk, descrevendo as técnicas empregadas para identificar estruturas internas do firmware, localizar arquivos incorporados e analisar padrões que indiquem compressão

ou criptografia. Além disso, são apresentados os resultados obtidos durante a varredura binária, bem como a interpretação dessas evidências no contexto da pesquisa.

A Figura 23 mostra a saída do comando “binwalk” sem nenhuma flag, na qual se identificam três arquivos comprimidos em áreas distintas do firmware.

Figura 23 – Retorno da execução do comando “binwalk BIOS.rom”



```

> binwalk BIOS.rom

/home/user/BIOS.rom
-----
DECIMAL          HEXADECIMAL      DESCRIPTION
-----
667016           0xA2D88          CRC32 polynomial table, little endian
668400           0xA32F0          SHA256 hash constants, little endian
870600           0xD48C8          LZMA compressed data, properties: 0x5D,
                    dictionary size: 16777216 bytes, compressed
                    size: 20903 bytes, uncompressed size: 41088
                    bytes
3854016          0x3ACEC0         SHA256 hash constants, little endian
4090712          0x3E6B58         LZMA compressed data, properties: 0x5D,
                    dictionary size: 16777216 bytes, compressed
                    size: 385169 bytes, uncompressed size:
                    1343504 bytes
4604216          0x464138         LZMA compressed data, properties: 0x5D,
                    dictionary size: 16777216 bytes, compressed
                    size: 1235154 bytes, uncompressed size:
                    5099536 bytes
-----

Analyzed 1 file for 85 file signatures (187 magic patterns) in 97.0 milliseconds

/home/user
>

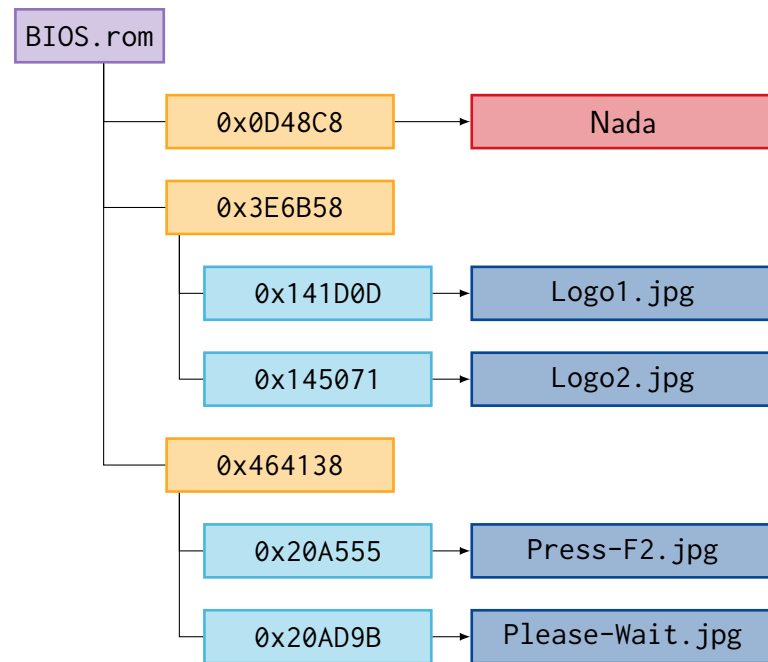
```

Fonte: Autor

O comando com a flag “-e” extrai arquivos para os quais o Binwalk oferece suporte. No caso de arquivos LZMA, a ferramenta realiza tanto a extração quanto a descompactação. Os binários extraídos recebem como nome o endereço hexadecimal correspondente ao início do arquivo compactado, conforme apresentado na figura anterior.

Embora identifique diversos executáveis no firmware, o Binwalk não possui capacidade nativa para extraí-los. A Figura 24 apresenta a árvore de extrações gerada a partir do arquivo original, evidenciando apenas os elementos cujo formato é reconhecido e manipulado pela ferramenta.

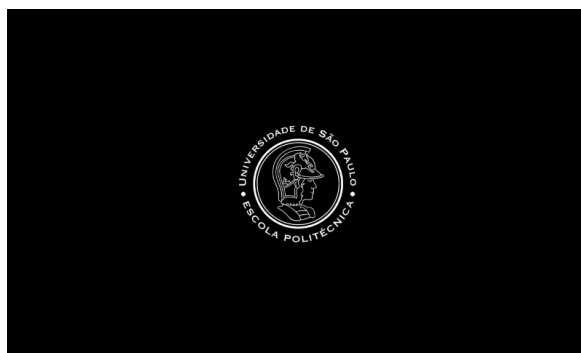
Figura 24 – Diagrama de árvore contendo os arquivos extraídos pelo Binwalk



Fonte: Autor

Entre os arquivos de imagem obtidos, aqueles que fornecem a maior quantidade de informações são **Logo1.jpg** e **Logo2.jpg**, pois seu conteúdo sugere que o firmware foi reaproveitado ou que ainda pode operar em um modo alternativo ao modo normal de funcionamento (modo de fábrica), conforme ilustrado nas Figuras 25a e 25b. Ressalta-se que a identidade visual original foi substituída pela da Escola Politécnica da Universidade de São Paulo, em conformidade com as restrições estabelecidas no NDA.

Figura 25 – Imagens obtidas através da ferramenta Binwalk (Censuradas)



(a) Arquivo “Logo1.jpg”



(b) Arquivo “Logo2.jpg”

Fonte: Autor

5.2.4 Investição com UEFITool

Esta seção aborda o processo de obtenção de informações por meio da ferramenta Binwalk, descrevendo as técnicas empregadas para identificar estruturas internas do firmware, localizar arquivos incorporados e analisar padrões que indiquem compressão ou criptografia. Além disso, são apresentados os resultados obtidos durante a varredura binária, bem como a interpretação dessas evidências no contexto da pesquisa.

5.2.4.1 Análise da Estrutura da Imagem

A Figura 26 mostra a estruturação da imagem UEFI do firmware obtido. Cada uma das regiões é responsável por armazenar arquivos e informações específicas para o funcionamento do programa, conforme detalhado nas seções a seguir.

Figura 26 – Principais seções do arquivo BIOS.rom

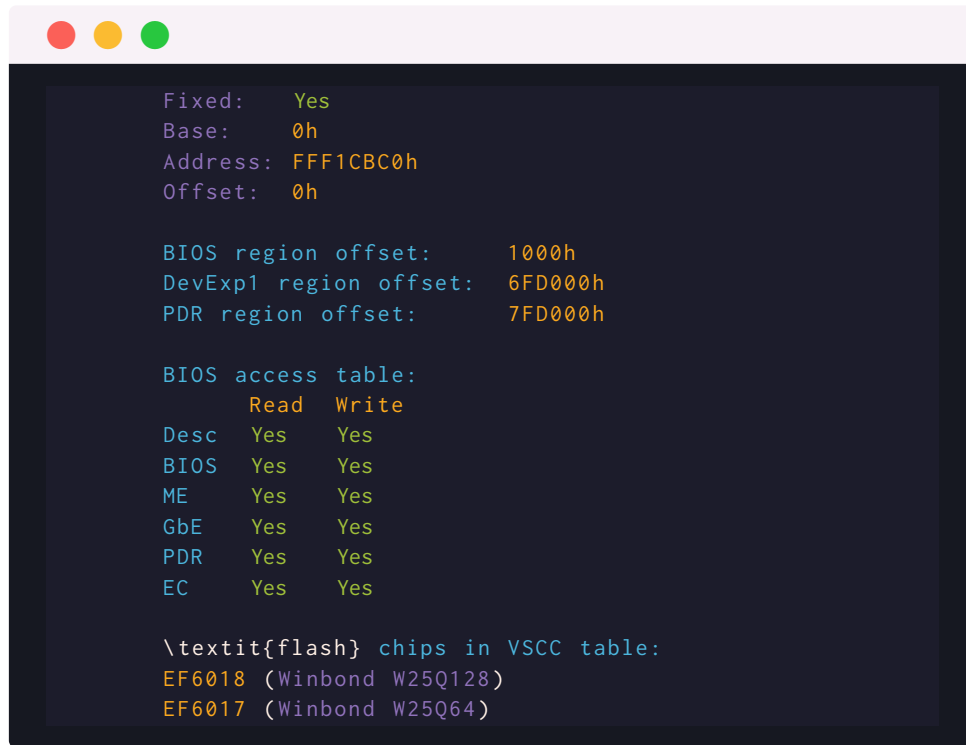
Structure			
Name	Action	Type	Subtype
▼ Intel image		Image	Intel
Descriptor region		Region	Descriptor
▶ BIOS region		Region	BIOS
DevExp1 region		Region	DevExp1
PDR region		Region	PDR
Padding		Padding	Empty (FFh)

Fonte: Autor

5.2.4.1.1 Região Descriptor

A região denominada “Descriptor” contém informações relativas ao firmware, incluindo endereços base, regras de acesso, leitura e escrita. A Figura 27 apresenta as principais informações extraídas dessa área, bem como o significado de cada uma delas.

Figura 27 – Informações presentes na região “Descriptor”



```

Fixed:    Yes
Base:     0h
Address:  FFF1CBC0h
Offset:   0h

BIOS region offset:    1000h
DevExp1 region offset: 6FD000h
PDR region offset:    7FD000h

BIOS access table:
      Read  Write
Desc  Yes   Yes
BIOS  Yes   Yes
ME    Yes   Yes
GbE   Yes   Yes
PDR   Yes   Yes
EC    Yes   Yes

\textit{flash} chips in VSCC table:
EF6018 (Winbond W25Q128)
EF6017 (Winbond W25Q64)

```

Fonte: Autor

Como pode ser observado, tanto o processador quanto o módulo de segurança possuem permissão de leitura e escrita em todas as regiões do firmware. Além disso, é possível identificar os endereços das demais regiões, bem como os modelos de memória *flash* compatíveis com o sistema.

5.2.4.1.2 Região DevExp1

A região DevExp1 corresponde a uma das áreas de expansão definidas pelo fabricante no *Descriptor* e é utilizada para armazenar dados complementares específicos da plataforma. Essa região abriga o armazenamento de variáveis UEFI, incluindo entradas de *boot*, variáveis de atalhos, registros de funcionamento do firmware e dados persistentes de configuração.

5.2.4.1.3 Região PDR

A região PDR constitui um segmento opcional definido pelo fabricante, destinado ao armazenamento de informações específicas da plataforma. Esse espaço pode conter chaves criptográficas proprietárias, dados relacionados a mecanismos de anti-furto, configurações internas do sistema, calibrações de manufatura, informações de licenciamento, registros industriais e outros blocos protegidos.

5.2.4.1.4 Região BIOS

A região BIOS abriga o firmware principal de inicialização, estruturado como volumes UEFI contendo módulos PEI e DXE, o gerenciador de *boot*, microcódigos de CPU e *drivers* de *chipset*. Essa região também pode incluir o armazenamento de variáveis UEFI, implementações de protocolos fundamentais e *drivers* essenciais para inicializar o antes da entrega do controle ao sistema operacional.

5.2.4.2 Busca de Módulos Proprietários

Por meio de uma análise manual, identificou-se um módulo que aparenta ter sido desenvolvido integralmente pela corporação, não se tratando, portanto, de uma modificação de um *driver* DXE convencional, apresentado na Figura 28. Além de apresentar um nome em português, a varredura realizada com o comando `strings` revelou termos exibidos durante a sequência de *boot* na tela principal, reforçando a hipótese de que se trata de um módulo proprietário, como mostra a Figura 29.

Figura 28 – *Driver* “SeguroDxe” localizado na região BIOS

▶ PhPlatformSiSmmCodeCheckSmm	SMM module	PhPlatformSiSmmCodeCheckSmm
▶ PlatformPreInit	DXE driver	PlatformPreInit
▶ SystemHddPwdSmm	SMM module	SystemHddPwdSmm
▶ I2cTouchPanelDxe	DXE driver	TouchDriverDxe
▶ EB39785A-1773-41B1-B201-2788950BF663	SMM module	PhPlatformEccHandlerSmm
▶ 8597940F-2C23-4A43-8656-C90E6512D628	DXE driver	PhPlatformEccHandlerDxe
▶ 8BDEBE8B-D6CA-C748-AD64-6B5D9FF0E59F	DXE driver	SeguroDxe
▶ 221521AE-0A35-44CD-80B5-5AEDBB770B1D	Freeform	
▶ SmmBaseOnSmmBase2Thunk	DXE driver	SmmBaseOnSmmBase2Thunk
▶ SmmBaseHelper	SMM module	SmmBaseHelper
▶ SmmControl20nSmmControlThunk	DXE driver	SmmControl20nSmmControlThunk
▶ SmbiosDxe	DXE driver	SmbiosDxe
▶ MiscSubclass	DXE driver	MiscSubclass
▶ DevicePathDxe	DXE driver	DevicePathDxe
▶ EdkShell	Application	
Volume free space		
▶ 0BBY	Code	

Fonte: Autor

Figura 29 – Texto legível presente no binário do arquivo “SeguroDxe.efi”

```

/home/user - fish
/home/user
> strings -ael SeguroDxe.efi
o existe unidade v
lida para boot !
Boot sendo realizado
Erro ao ler o arquivo de configura
Arquivo de configura
o inv
lido !
Erro ao ler o arquivo  bootloader !
Arquivo  bootloader inv
lido !
Boot sendo realizado
Verificando o Sistema Operacional
Error ao ler o arquivo do Sistema Operacional !
Arquivo de Sistema Operacional inv
lido !

/home/user
>

```

Fonte: Autor

Também identifica-se a existência de uma EDK *shell*, como indica a Figura 30.

Figura 30 – Aplicação “EdkShell” localizada na região BIOS

PhPlatformSmmCodeCheckSmm	SMM module	PhPlatformSmmCodeCheckSmm
PlatformPreInit	DXE driver	PlatformPreInit
SystemHddPwdSmm	SMM module	SystemHddPwdSmm
I2cTouchPanelDxe	DXE driver	TouchDriverDxe
EB39785A-1773-41B1-B201-2788950BF663	SMM module	PhPlatformEccHandlerSmm
8597940F-2C23-4A43-8656-C90E6512D628	DXE driver	PhPlatformEccHandlerDxe
8BDEBE8B-D6CA-C748-AD64-6B5D9FF0E59F	DXE driver	SeguroDxe
Z21521AE-0A35-44CD-80B5-5AEDB8770B1D	Freeform	
SmmBaseOnSmmBase2Thunk	DXE driver	SmmBaseOnSmmBase2Thunk
SmmBaseHelper	SMM module	SmmBaseHelper
SmmControl2OnSmmControlThunk	DXE driver	SmmControl2OnSmmControlThunk
SmbiosDxe	DXE driver	SmbiosDxe
MiscSubclass	DXE driver	MiscSubclass
DevicePathDxe	DXE driver	DevicePathDxe
EdkShell	Application	
Volume free space		
Volume free space		
0BBY	Code	

Fonte: Autor

A EDK *Shell* atua como um interpretador de comandos que opera diretamente sobre os serviços de *boot* e *runtime* da arquitetura UEFI. Ela fornece um ambiente de execução anterior ao sistema operacional, permitindo o acesso a dispositivos, a manipulação de sistemas de arquivos FAT, a execução de aplicativos UEFI no formato PE/COFF e a interação com protocolos internos do firmware. Destinada fundamentalmente ao desenvolvimento e à depuração, a EDK *Shell* inclui comandos e funcionalidades adicionais voltados a testes, análise e validação de componentes UEFI.

6 ANÁLISE DE VULNERABILIDADES

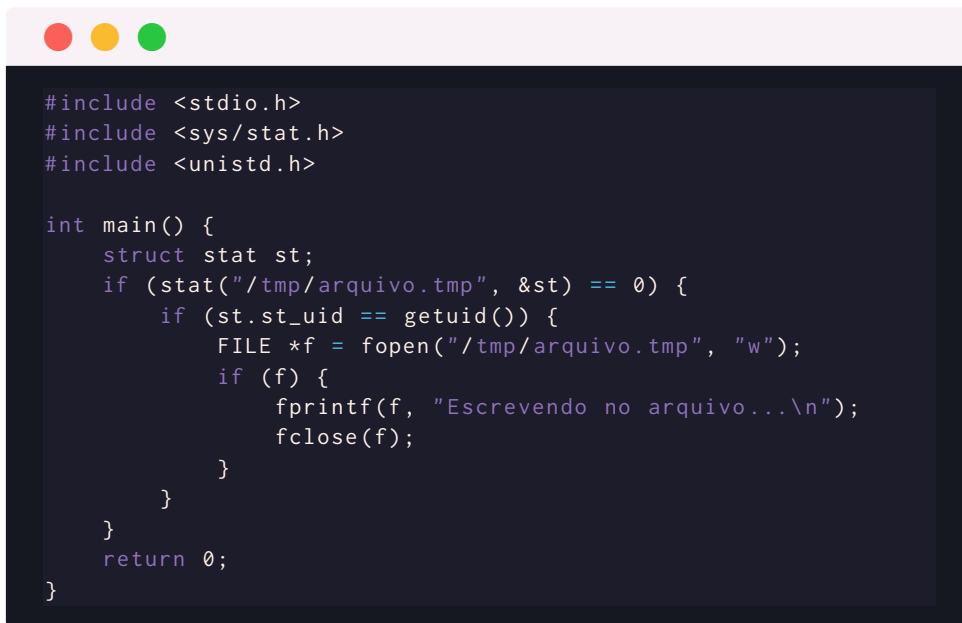
Este capítulo sintetiza os resultados das análises apresentadas no capítulo anterior. Com base na modelagem de ameaças previamente desenvolvida, são formuladas hipóteses sobre possíveis vetores de ataque, bem como avaliada a viabilidade de sua execução.

A Figura 32 apresenta a arquitetura do circuito de inicialização do sistema. A seta em vermelho, localizada entre “Desliga circuito da BIOS” e “Liga todos os circuitos”, indica um vetor de exploração potencial associado a uma condição de corrida, especificamente um ataque do tipo TOCTOU, identificado pela sigla CWE-367.

O CWE-367, denominado TOCTOU, descreve uma categoria de vulnerabilidade associada a condições de corrida em que um recurso é verificado em um determinado instante, mas utilizado em outro. Nesse intervalo temporal, um agente malicioso pode alterar o estado do recurso, levando o sistema a operar com informações desatualizadas ou incorretas. Trata-se de uma falha típica em sistemas concorrentes ou em ambientes nos quais múltiplas entidades possuem acesso simultâneo a arquivos, dispositivos ou estruturas de dados compartilhadas ([The MITRE Corporation, 2025](#)).

A vulnerabilidade ocorre porque a suposição de que o estado do recurso permanece inalterado entre a verificação e o uso nem sempre é válida. Em sistemas multitarefa, operações como verificações de permissões, leitura de metadados e validação de integridade podem ser rapidamente invalidadas caso o recurso seja modificado antes da sua utilização efetiva. Assim, um invasor pode explorar essa janela de tempo para substituir, remover ou modificar o recurso, fazendo com que o sistema execute ações potencialmente inseguras sob a falsa impressão de que o estado inicial permanece o mesmo. A Figura 31 mostra um código que exemplifica a exploração desta vulnerabilidade: o programa verifica se o arquivo existe e se pertence ao usuário (`stat()` – “*Time-of-Check*”), e depois o usa para escrita (`fopen()` – “*Time-of-Use*”).

Figura 31 – Exemplo de vulnerabilidade de condição de corrida



```
#include <stdio.h>
#include <sys/stat.h>
#include <unistd.h>

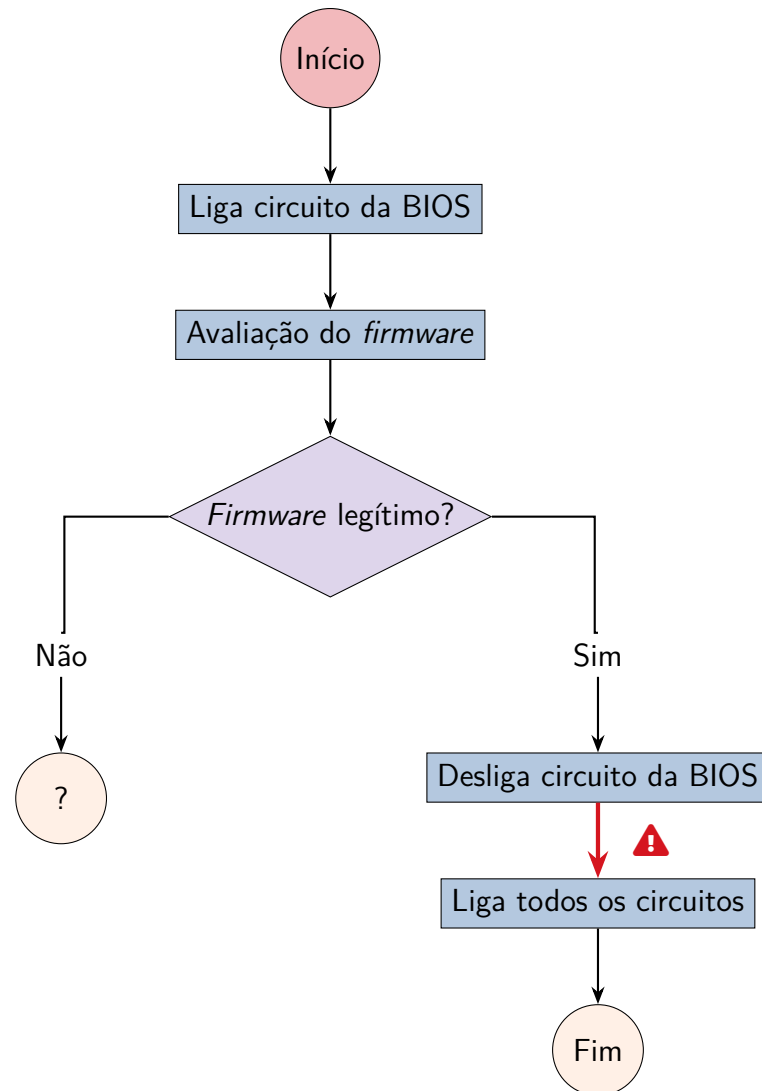
int main() {
    struct stat st;
    if (stat("/tmp/arquivo.tmp", &st) == 0) {
        if (st.st_uid == getuid()) {
            FILE *f = fopen("/tmp/arquivo.tmp", "w");
            if (f) {
                fprintf(f, "Escrevendo no arquivo...\n");
                fclose(f);
            }
        }
    }
    return 0;
}
```

Fonte: Autor

Cenários comuns incluem manipulação de arquivos temporários, links simbólicos, descritores de arquivos e elementos de configuração acessados por múltiplos processos. Em ambientes de firmware ou sistemas embarcados, um ataque TOCTOU pode permitir que um componente malicioso modifique um artefato após sua validação, mas antes de sua execução, possibilitando a introdução de código não autorizado sem acionar mecanismos de defesa baseados em verificações prévias.

Os impactos associados a essa vulnerabilidade variam conforme o contexto, podendo incluir escalonamento de privilégios, desvio de fluxo de execução, modificação não autorizada de dados e *bypass* de verificações de segurança. A mitigação envolve reduzir ou eliminar a janela entre verificação e uso, adotar operações atômicas, empregar bloqueios de acesso (*locking*) apropriados ou redesenhar o fluxo de execução para que a verificação e o uso do recurso sejam realizados como uma única operação atômica.

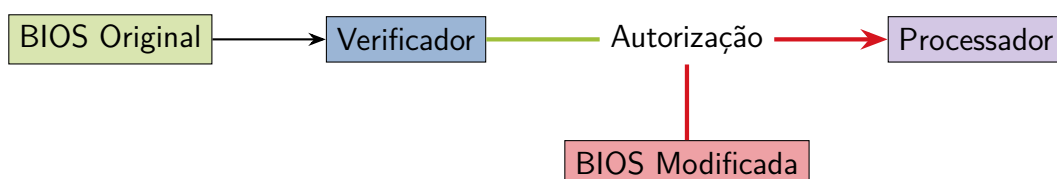
Figura 32 – Fluxograma correspondente ao funcionamento observado do caixa



Fonte: Autor

No contexto do cenário do sistema em estudo, essa condição abre a possibilidade de carregar um firmware diferente daquele previamente analisado pelo módulo de segurança, no intervalo entre a finalização da verificação da BIOS e o religamento dos demais circuitos. A Figura 33 apresenta o diagrama de blocos que ilustra o modo como essa exploração pode ser conduzida.

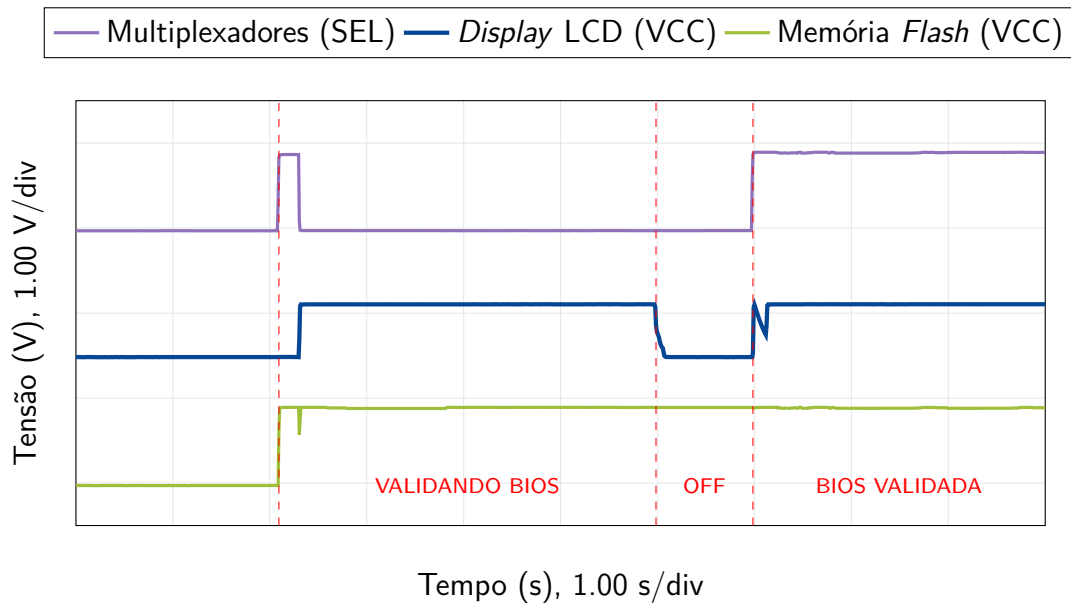
Figura 33 – Exploração de condição de corrida



Fonte: Autor

Utilizando um osciloscópio, foi possível comprovar a existência de uma janela de tempo, conforme ilustrado na Figura 34. Durante o intervalo indicado como “OFF”, é viável realizar o chaveamento do firmware, pois a placa-mãe encontra-se desligada - evidenciado pelo sinal do *display* LCD - e ainda não ocorreu a multiplexação entre o módulo de segurança e o processador.

Figura 34 – Monitoramento de sinais de componentes no osciloscópio



Fonte: Autor

Além disso, foram identificados fatores que podem ser explorados por um agente malicioso para tornar o ataque mais bem sucedido:

Tabela 5 – Possíveis vetores de exploração identificados, com impacto e dificuldade técnica associados

	Descrição	Impacto	Dificuldade
1	Uso de pinos não conectados para estabelecer novas conexões no circuito.	Moderado	Moderada
2	Inserção de um oitavo terminal no conector CN12 para criar um novo ponto de conexão externa.	Moderado	Moderada
3	Inserção de um módulo externo conectado aos terminais J1 e CN12.	Alto	Alta

Continua

	Descrição	Impacto	Dificuldade
4	Uso de uma FPGA para realizar escrita em tempo de execução na memória <i>flash</i> , no intervalo entre a verificação e o carregamento do firmware.	Alto	Alta
5	Uso de memória com capacidade duplicada para armazenar duas versões distintas do firmware de <i>boot</i> , alternando entre ambas por meio do chaveamento do bit mais significativo.	Alto	Alta
6	Modificação do firmware de <i>boot</i> para carregar a aplicação “EdkShell”.	Alto	Alta
7	Modificação do <i>driver</i> “SeguroDxe.efi” para interceptar a comunicação entre módulo de segurança e periféricos.	Alto	Alta

O projeto de pesquisa descrito neste documento explora os vetores 1, 2 e 3, uma vez que emprega parte significativa do próprio circuito para a execução do ataque. As possibilidades relacionadas aos itens 4 e 5 são discutidas na Seção 9.3.

A classificação de impacto e dificuldade técnica atribuída aos vetores de exploração reflete os riscos e as exigências práticas associadas a cada abordagem.

Os itens 1 e 2 apresentam impacto moderado, pois permitem estender o circuito e introduzir novos pontos de acesso sem alterações estruturais profundas; contudo, demandam níveis distintos de habilidade manual para garantir conexões seguras e discretas.

O item 3 possui impacto alto, já que a inserção de um módulo externo conectado aos terminais J1 e CN12 habilita interferência direta no processo de inicialização, e sua dificuldade se deve ao fato do reaproveitamento da infraestrutura já existente, que precisa passar por modificações que exigem um conhecimento técnico maior.

Os itens 4 e 5 envolvem técnicas avançadas, classificando-se, portanto, como vetores de alto impacto e alta dificuldade técnica, tanto pela complexidade da execução quanto pelo risco de danos ao circuito ou perda de confiabilidade do processo de *boot*.

Os itens 6 e 7 dependem do êxito na exploração da condição de corrida no circuito, bem como da realização de engenharia reversa do firmware, uma vez que requer o mapeamento e a modificação de elementos em baixo nível para possibilitar o carregamento da

aplicação. Trata-se de uma abordagem substancialmente mais complexa em termos de dificuldade técnica, mas que apresenta impacto elevado, pois pode permitir a execução de código não autorizado.

7 EXPLORAÇÃO

A partir das informações coletadas no Capítulo 6, é possível avançar para a execução de testes e a levantamento e validação de hipóteses. Este capítulo abrange os testes realizados em e firmware, voltados à tentativa de exploração da janela de tempo existente entre a verificação e o uso da BIOS.

7.1 ETAPA 01 - ALTERAÇÃO DE *BYTES* DE TEXTO NO FIRMWARE

Trata-se de um teste simples da modificação da região PDR do arquivo binário que não afeta a execução do programa, mas a verificação de integridade do arquivo. A partir desse teste, pode-se formular hipóteses sobre como o módulo de segurança atua. As Figuras 35a e 35b mostram o trecho escolhido, no arquivo original e no arquivo modificado, respectivamente.

Figura 35 – Trecho do Arquivo Binário Escolhido para Alteração

007FCF74	FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF	007FCF74	FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
007FCF88	FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF	007FCF88	FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
007FCF9C	FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF	007FCF9C	FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
007FCFB0	FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF	007FCFB0	FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
007FCFC4	FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF	007FCFC4	FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
007FCFD8	FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF	007FCFD8	FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
007FCFEC	FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF	007FCFEC	FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
007FD000	42 52 32 30 32 34 FF FF FF FF FF FF FF FF FF FF	007FD000	4C 41 52 43 32 35 FF FF FF FF FF FF FF FF FF FF
007FD014	FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF	007FD014	FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
007FD028	FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF	007FD028	FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
007FD03C	FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF	007FD03C	FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
007FD050	FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF	007FD050	FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
007FD064	FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF	007FD064	FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
007FD078	FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF	007FD078	FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
007FD08C	FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF	007FD08C	FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF

(a) ASCII “BR2024” no Arquivo Original

(b) ASCII “LARC25” no Arquivo Modificado

(Fonte: Autor)

A escolha do trecho deve-se ao fato de este estar isolado no final do arquivo e não conter executáveis ou instruções ao redor. Como resultado do teste, o sistema de segurança retornou um código de erro no que diz respeito à validação do firmware. Isso sugere que a cadeia de verificação passa por uma etapa de avaliar o binário como um todo.

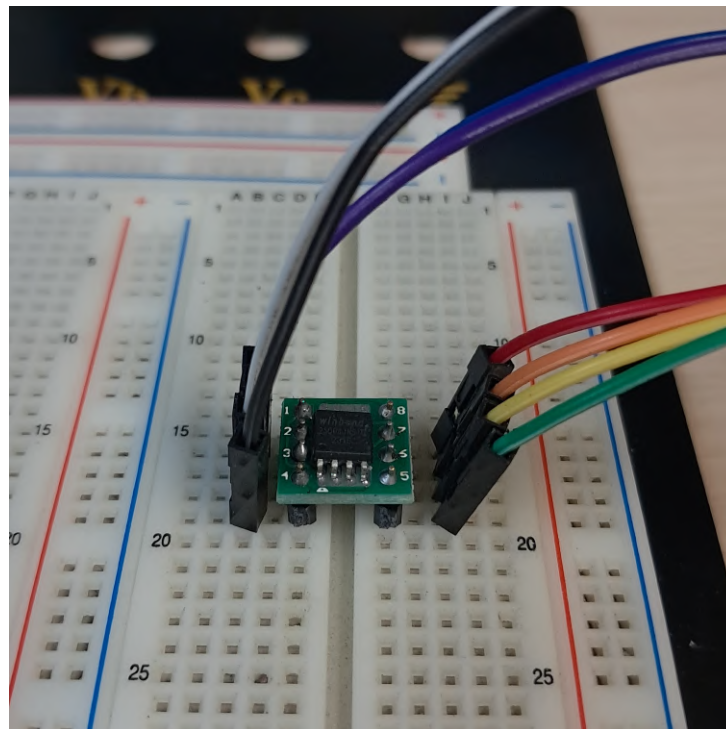
7.2 ETAPA 02 - INSERÇÃO DE MEMÓRIA EXTERNA NO BARRAMENTO

O teste em questão avalia a viabilidade de executar a cadeia de inicialização por meio de uma memória *flash* externa. Trata-se de uma etapa fundamental para que o ataque ocorra da forma esperada.

7.2.1 Uso de *Protoboard* e *Jumpers*

Inicialmente, empregaram-se *jumpers* para verificar se o módulo de segurança seria capaz de ler a memória a uma distância significativamente maior que a utilizada no circuito original, conforme ilustrado na Figura 36. O teste revelou-se infrutífero, indicando que o sinal pode perder integridade ou comportar-se como uma linha de transmissão em virtude da alta frequência de operação.

Figura 36 – Configuração utilizada para *boot* via *protoboard*

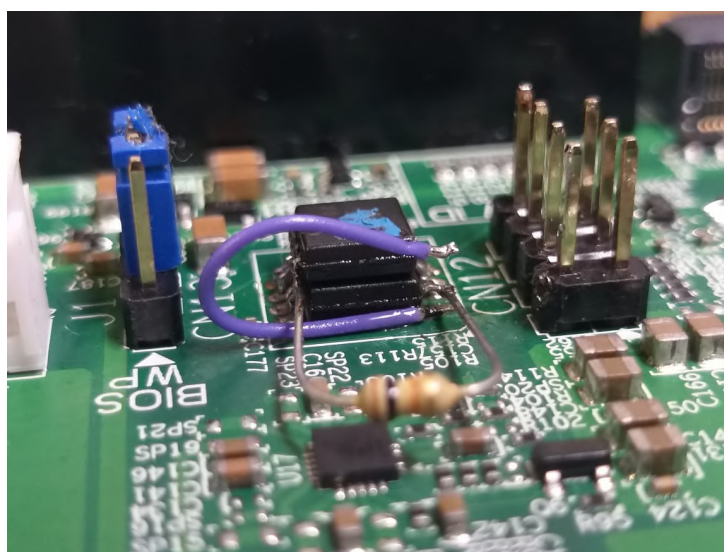


Fonte: Autor

7.2.2 Soldagem de Memória Sobre a Original

Para avaliar se a distância entre a memória e o módulo de segurança influenciava o processo de leitura, procedeu-se à soldagem de uma segunda memória *flash* contendo o firmware original, posicionada o mais próximo possível da memória já instalada, conforme mostrado na Figura 37.

Figura 37 – Configuração utilizada para *boot* via inserção direta no barramento



Fonte: Autor

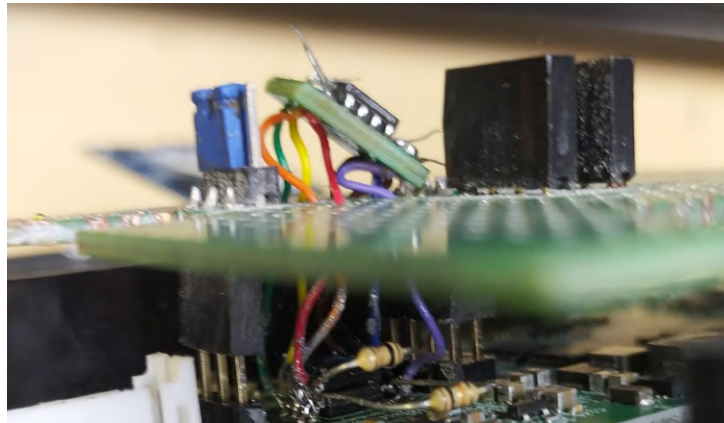
O terminal 1 da memória original foi dessoldado, levantado e conectado ao terminal de alimentação positiva, de modo a impedir sua leitura. Em seguida, o *pad* correspondente na placa foi ligado ao terminal 1 da memória inserida por meio de um pequeno fio.

O teste foi bem-sucedido, e a verificação do firmware ocorreu normalmente. Para confirmar que a leitura estava sendo realizada de fato a partir da memória inserida, esta foi removida, carregada com um firmware alterado e reinserida no barramento. Como esperado, a validação falhou, uma vez que o binário modificado não atende aos requisitos de integridade.

7.2.3 Verificação de Distâncias

Considerando o êxito do teste anterior, procedeu-se à soldagem de outra memória *flash* utilizando o mesmo método, porém conectada ao barramento por meio de fios curtos, conforme ilustrado na Figura 38.

Figura 38 – Configuração utilizada para *boot* via inserção por fios no barramento



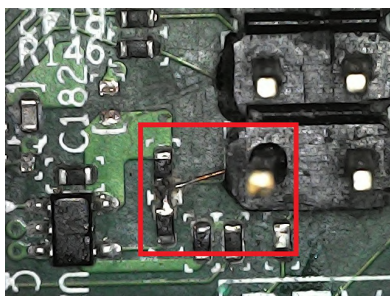
Fonte: Autor

Esse teste também foi bem-sucedido, indicando a viabilidade de desenvolver uma placa auxiliar destinada à execução do ataque, desde que os fios associados aos sinais de dados mantenham comprimento inferior a aproximadamente cinco centímetros, de modo a preservar a integridade dos sinais.

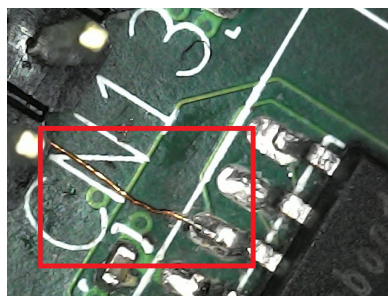
7.3 ETAPA 03 - CONSTRUÇÃO DE TRILHAS NA PLACA

Com o objetivo de minimizar a distância das conexões, explorou-se o pino desconectado no barramento “BIOS WP” e o espaço disponível no conector “CN12” para estabelecer novas ligações. Conforme mostrado na Figura 12, os sinais correspondentes aos pinos 1, 3 e 7 não apresentam roteamento para o barramento “CN12”. Em razão disso, tais conexões foram criadas artificialmente.

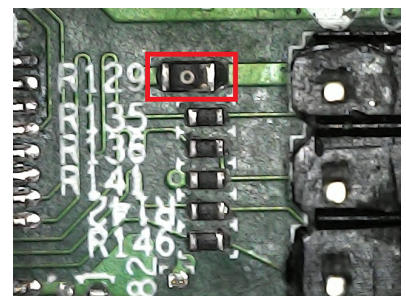
Figura 39 – Fotos sob microscópio das modificações realizadas no circuito



(a) Terminal 3



(b) Terminal 7



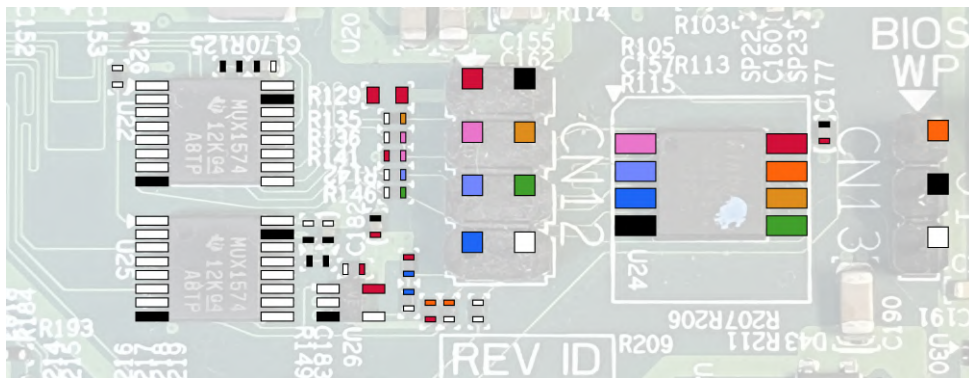
(c) Terminal 8

Fonte: Autor

A Figura 39 apresenta as modificações realizadas em cada um dos terminais. No caso do pino 3 (Figura 39a), inseriu-se um terminal falso, ao qual foi conectada a linha que segue para os resistores R160 e R170. Para o pino 7 (Figura 39b), estabeleceu-se uma ligação direta com o pino 1 do terminal “BIOS WP”. Já para o pino 8 (Figura 39c), o resistor R140 foi ressoldado de modo a permitir que a alimentação alcançasse o terminal 1 do conector “CN12”. Embora essa conexão pudesse ter sido feita por meio de um fio, optou-se pela utilização de um resistor para tornar a modificação menos perceptível.

Com essas alterações, o novo mapeamento de contatos para a memória *flash* corresponde ao apresentado na Figura 40.

Figura 40 – Novo arranjo de contatos para a memória *flash*

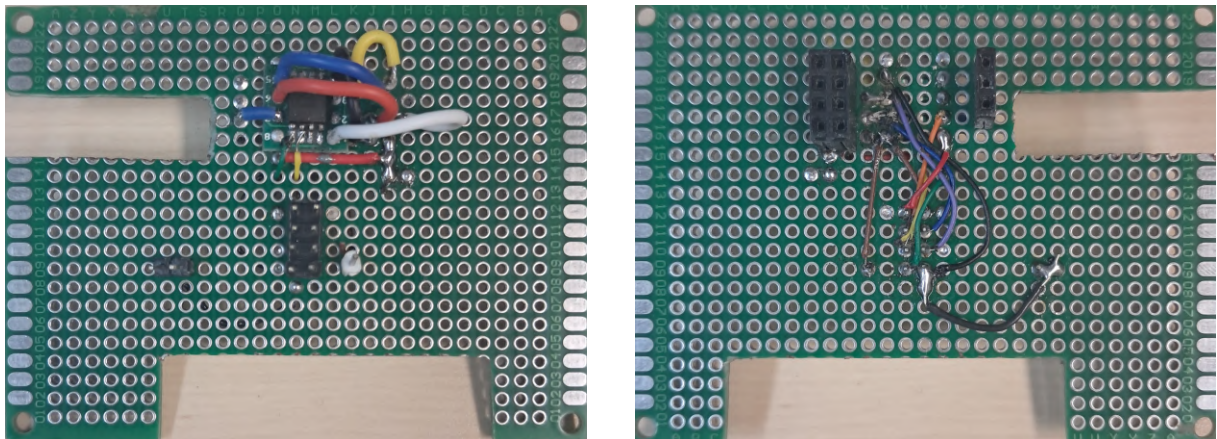


Fonte: Autor

7.4 ETAPA 04 - *BOOT* PELOS PINOS EXTERNOS

Com todos os pinos da memória *flash* acessíveis de forma mais prática, torna-se possível desenvolver um *shield* capaz de realizar o chaveamento entre diferentes versões de firmware. Um protótipo inicial foi construído utilizando conexões rentes à placa perfurada; contudo, observou-se perda de integridade do sinal. Em razão disso, elaborou-se um novo protótipo empregando fios curtos (inferiores a cinco centímetros) para realizar as ligações. A Figura 41 apresenta o circuito inicial, composto por uma única memória *flash* com suas conexões mapeadas para os pinos correspondentes na placa-mãe. Há também oito pinos destinados à gravação de firmware, caso necessário.

Figura 41 – Protótipo inicial para teste de *boot* pelos pinos externos



(a) Vista superior do *shield*

(b) Vista inferior do *shield*

Fonte: Autor

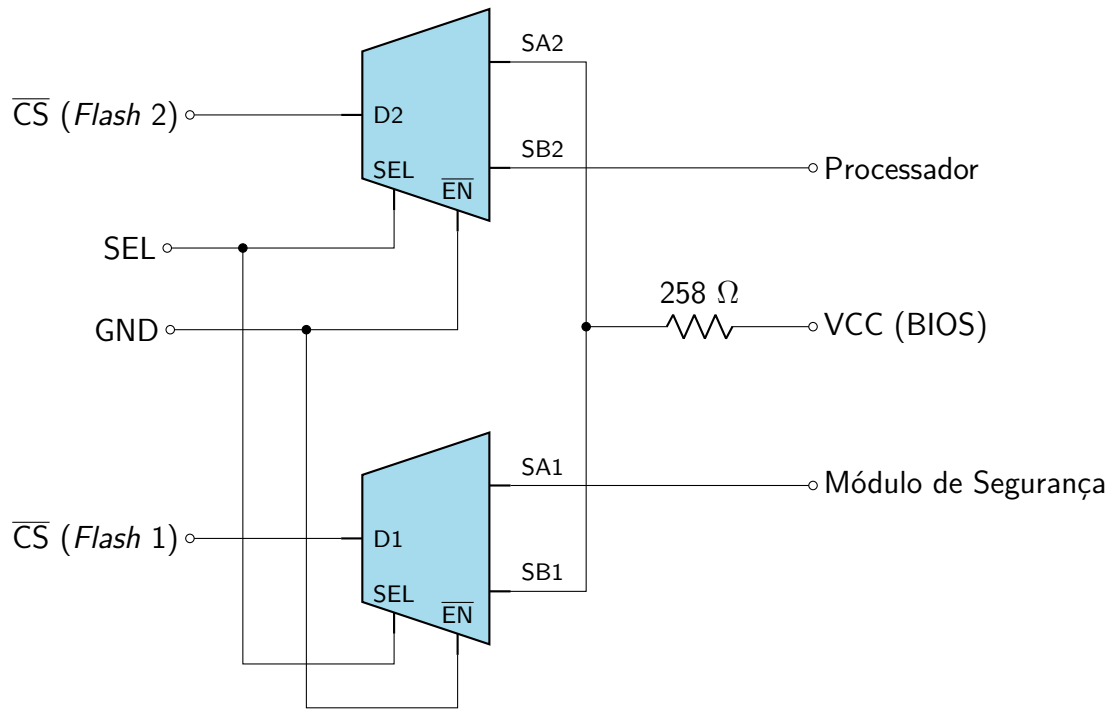
O *boot* foi bem-sucedido, permitindo que o ataque fosse conduzido de forma menos invasiva. Isso possibilitou a adição de componentes ao protótipo desenvolvido, em vez de exigir que fossem soldados diretamente na placa-mãe.

7.5 ETAPA 05 - CIRCUITO DE CHAVEAMENTO

Apesar de os multiplexadores presentes na placa-mãe indicarem o não uso de certos pinos, conforme apresentado na Seção 5.1.2.2.5, optou-se por empregar um multiplexador externo, do mesmo modelo, para realizar o chaveamento e tornar o ataque menos invasivo.

Dado o funcionamento do circuito integrado em questão, o controle pode ser implementado por meio de dois multiplexadores, cada um responsável pela ativação ou desativação de uma das memórias. A Figura 42 apresenta o circuito equivalente proposto.

Figura 42 – Esquema do circuito de multiplexação



Fonte: Autor

Considerando o sinal “SEL”, que controla os comutadores U22 e U25, tem-se que, durante a fase de verificação, as portas “SA1” e “SA2” permanecem ativas, de modo que a memória *flash* 1 é analisada enquanto a memória *flash* 2 permanece desabilitada. Já durante a fase de execução, o sinal “SEL” ativa as portas “SB1” e “SB2”, fazendo com que a memória *flash* 1 seja desativada e a memória *flash* 2 passe a ser executada pelo processador. O resistor presente no circuito corresponde a um resistor de *pull-up*, cujo valor máximo foi definido experimentalmente, de forma a garantir a inibição adequada das memórias.

7.6 ETAPA 06 - MONTAGEM DO PROTÓTIPO

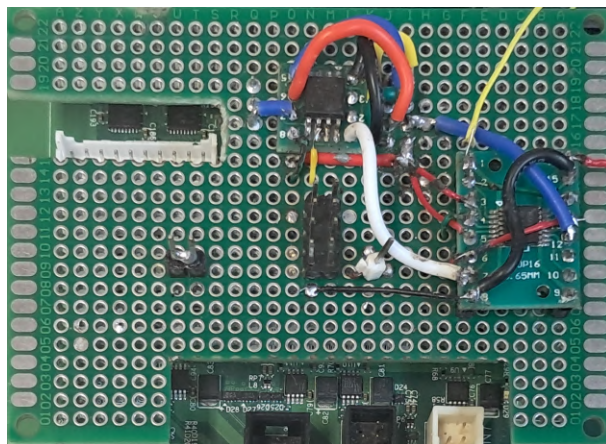
Como observado no mapeamento dos terminais dos multiplexadores (Figura 16), a alimentação desses componentes difere daquela utilizada pela memória *flash*. Além disso, é necessário conectar o pino 1 da memória *flash* da placa-mãe ao circuito, uma vez que ele deve ser encaminhado ao multiplexador, conforme ilustrado na Figura 42.

Dessa forma, além da inclusão do novo multiplexador no *shield*, este deve ser conectado tanto à alimentação quanto ao sinal de controle dos dois multiplexadores presentes na placa-mãe. A Figura 43 apresenta as modificações realizadas na placa-mãe para atender

7.7 ETAPA 07 - EXECUÇÃO DO ATAQUE

A Figura 45 mostra o circuito montado sobre a placa-mãe. Inicialmente, testou-se a execução da cadeia de *boot* com as duas memórias *flash* contendo o firmware íntegro para validar o funcionamento do circuito.

Figura 45 – Shield conectado à placa-mãe



Fonte: Autor

Com a validação bem-sucedida do teste, um firmware contendo a modificação descrita na Seção 7.1 foi gravado e o sistema inicializado. O módulo de segurança indicou que o firmware era autêntico - o que é esperado, uma vez que a verificação ocorre sobre a versão original - e, mesmo assim, autorizou a execução do firmware alterado. Essa situação é confirmada pelo fato de que a cadeia de inicialização é interrompida pelo processador, sinalizando uma possível inconsistência na assinatura do arquivo.

Apesar da interrupção promovida pelo processador, foi possível confirmar a hipótese de que o módulo de segurança do caixa eletrônico pode ser contornado. Esse resultado abre espaço para a exploração de outros vetores de ataque, especialmente em nível de firmware UEFI.

8 PÓS-EXPLORAÇÃO

A fase de pós-exploração tem como propósito avaliar o impacto potencial da violação do *boot* seguro após a inserção e comutação da memória *flash* externa. Uma vez confirmado que o equipamento aceitava a imagem adulterada no momento da transição entre as etapas de inicialização, verificou-se que o invasor obtém capacidade de modificar componentes críticos do firmware sem acionar mecanismos internos de detecção.

Os experimentos mostraram que, após a conclusão bem-sucedida do ataque, torna-se possível executar código arbitrário antes da carga do sistema operacional, comprometendo a cadeia de confiança e permitindo intervenções persistentes. Tal condição permitiria, em um cenário real, a manipulação de periféricos, a interceptação de dados sensíveis ou a implantação de implantes maliciosos de difícil detecção.

Adicionalmente, observou-se que a viabilidade do ataque independe de acesso ao equipamento em funcionamento, uma vez que o circuito pode ser removido e manipulado externamente. Isso amplia de forma significativa o impacto potencial da vulnerabilidade, pois há a possibilidade de ocorrência em um ambiente controlado, no qual um possível atacante dispõe de tempo e recursos para aprofundar o comprometimento.

Por fim, a análise evidenciou a ausência de mecanismos de proteção física e lógica capazes de impedir a substituição dinâmica da memória, reforçando a necessidade de mecanismos redundantes de validação de integridade e monitoramento do barramento SPI.

9 CONSIDERAÇÕES FINAIS

9.1 CONCLUSÕES DO PROJETO DE FORMATURA

Este trabalho analisou a possibilidade de violação do processo de *boot* seguro de um caixa eletrônico em transição entre BIOS e UEFI por meio da multiplexação da memória *flash* responsável pelo firmware. A investigação experimental permitiu demonstrar que a arquitetura do equipamento apresenta janelas de vulnerabilidade entre as etapas de verificação e uso, permitindo a introdução de uma imagem adulterada sem que os mecanismos internos de proteção detectem o evento.

As etapas conduzidas (mapeamento físico do circuito, análise dos componentes, extração e engenharia reversa do firmware, identificação de vulnerabilidades e teste prático da exploração) forneceram evidências de que o arranjo atual da placa possibilita intervenções externas capazes de comprometer a cadeia de confiança. Essa constatação ressalta a relevância de estudos que discutam riscos associados ao de ATMs.

9.2 CONTRIBUIÇÕES

O presente projeto apresenta três principais contribuições. A primeira consiste em uma análise empírica de um ATM real, acompanhada de documentação detalhada dos componentes envolvidos no processo de inicialização. A segunda refere-se à demonstração prática de um vetor de ataque estruturado, descrevendo cada etapa necessária para a substituição dinâmica da memória e a consequente adulteração do firmware. Por fim, a terceira contribuição envolve uma discussão técnica sobre limitações do modelo de segurança, com ênfase em fragilidades estruturais relacionadas ao barramento SPI e à ausência de mecanismos de validação contínua de integridade.

Essas contribuições podem fundamentar revisões futuras de arquitetura, além de apoiar o desenvolvimento de mecanismos adicionais de proteção física e lógica em sistemas embarcados utilizados por instituições financeiras.

9.3 PERSPECTIVAS DE CONTINUIDADE

A continuidade deste trabalho pode seguir diferentes direções. Uma possibilidade consiste no desenvolvimento de um módulo de proteção física destinado ao monitoramento do barramento SPI. Outra linha de investigação envolve a aplicação de técnicas de medição ativa da integridade da memória durante o processo de inicialização. Adicionalmente, podem ser estudadas contramedidas baseadas em criptografia ou em mecanismos de detecção de anomalias elétricas no acesso à memória. Por fim, destaca-se também a expansão do ataque demonstrado, bem como a exploração dos vetores discutidos no Capítulo 6, com o objetivo de aprofundar o entendimento das vulnerabilidades identificadas.

Pretende-se dar continuidade a essas linhas de investigação na pós-graduação, com ênfase na exploração do firmware UEFI em maior profundidade, visando romper a cadeia de confiança por completo. Além disso, propõe-se o desenvolvimento de uma solução viável para o problema identificado, acompanhado da avaliação de sua efetividade.

REFERÊNCIAS

ALTIUM LLC. *Altium Designer - PCB Design Software*. Altium. Disponível em: <https://www.altium.com/altium-designer>. Acesso em: 7 out. 2025.

ARDUINO. *Arduino IDE*. Disponível em: <https://docs.arduino.cc/software/ide/>. Acesso em: 7 out. 2025.

ARDUINO. *Nano / Arduino Documentation*. Disponível em: <https://docs.arduino.cc/hardware/nano/>. Acesso em: 7 out. 2025.

BANCO CENTRAL DO BRASIL. *Quantidade de ATMs no Brasil*. Jul. 2017. Disponível em: <https://dadosabertos.bcb.gov.br/dataset/24911-quantidade-de-atms-no-brasil>. Acesso em: 1 maio 2025.

BANCO CENTRAL DO BRASIL. *Quantidade de saques por canal presencial : ATM*. Jul. 2017. Disponível em: <https://dadosabertos.bcb.gov.br/dataset/25185-quantidade-de-saques-por-canal-presencial---atm>. Acesso em: 1 maio 2025.

BANCO CENTRAL DO BRASIL. *Relatório de Economia Bancária - 2022*. 2022. Disponível em: <https://www.bcb.gov.br/content/publicacoes/relatorioeconomiabancaria/reb2022p.pdf>.

BLACK HAT USA. *Jackpotting Automated Teller Machines Redux*. 2010. Disponível em: <https://www.blackhat.com/html/bh-us-10/bh-us-10-archives.html#Jack>. Acesso em: 17 ago. 2025.

FLASHROM. *Flashrom*. Disponível em: <https://github.com/flashrom/flashrom>. Acesso em: 7 out. 2025.

FRAILE, Marlon *et al.* *Using Attack-Defense Trees to Analyze Threats and Countermeasures in an ATM: A Case Study*. In: HORKOFF, Jennifer; JEUSFELD, Manfred A.; PERSSON, Anne (ed.). **The Practice of Enterprise Modeling**. 2016, p. 326–334. ISBN 978-3-319-48393-1. DOI: [10.1007/978-3-319-48393-1_24](https://doi.org/10.1007/978-3-319-48393-1_24). Acesso em: 4 nov. 2025.

LONGSOFT. *UEFITool*. Disponível em: <https://github.com/LongSoft/UEFITool>. Acesso em: 7 out. 2025.

NANJING QINHENG MICROELECTRONICS. *USB Bus Convert Chip CH341*. Disponível em: https://extra.onetransistor.eu/download/datasheet/CH341_EN.pdf. Acesso em: 7 out. 2025.

RAMIREZ, J. *An Anomaly Behavior Analysis Methodology for the Internet of Things: Design, Analysis and Evaluation*. 2017. Tese (Doutorado) – University of Arizona. Disponível em: https://repository.arizona.edu/bitstream/handle/10150/625581/azu_etd_15658_sip1_m.pdf. Acesso em: 17 ago. 2025.

REFIRM LABS. *Binwalk*. Disponível em: <https://github.com/ReFirmLabs/binwalk>. Acesso em: 7 out. 2025.

S-MANUALS. *Marking of Electronic Components, SMD Codes*. Disponível em: <https://www.s-manuals.com/smd/ve>. Acesso em: 5 maio 2025.

SHARMA, P.; GUPTA, R. *Design and Development for Detection and Prevention of ATM Skimming Frauds*. **International Journal of Advanced Computer Science and Applications**, v. 11, n. 7, 2020. Disponível em: <https://www.researchgate.net/publication/339608837>. Acesso em: 7 out. 2025.

SHUKLA, S.; KUMAR, A.; SINGH, R. *Hand Me Your PIN!: Inferring ATM PINs of Users Typing with a Covered Hand*. In: PROCEEDINGS of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 2021. Disponível em: <https://arxiv.org/abs/2110.08113>. Acesso em: 7 out. 2025.

TADEUSZ, W.; NOWAK, M. *Digital Robbery: ATM Hacking and Implications*. 2021. DOI: [10.1007/978-3-030-70706-4](https://doi.org/10.1007/978-3-030-70706-4). Acesso em: 7 out. 2025.

TEXAS INSTRUMENTS. *TMUX1574 Low Capacitance, 2:1 (SPDT) 4 Channel, Powered Off Protected Switch with 1.8 V Logic*. 2018. Disponível em: <https://www.ti.com/lit/ds/symlink/tmux1574.pdf>. Acesso em: 3 jun. 2025.

THE MITRE CORPORATION. *CWE-367: Time-of-check Time-of-use (TOCTOU) Race Condition*. 2025. Disponível em: <https://cwe.mitre.org/data/definitions/367.html>. Acesso em: 16 set. 2025.

THE PENETRATION TESTING EXECUTION STANDARD. *High Level Organization of the Standard*. Ago. 2014. Disponível em: http://www.pentest-standard.org/index.php/Main_Page. Acesso em: 30 jun. 2025.

WINBOND. *1.8V 64M-Bit Serial Flash Memory With Dual, Quad SPI*. 2025. Disponível em: <https://www.winbond.com/resource-files/W25Q64JW%20RevF%2009262025%20Plus.pdf>. Acesso em: 10 nov. 2025.

APÊNDICES

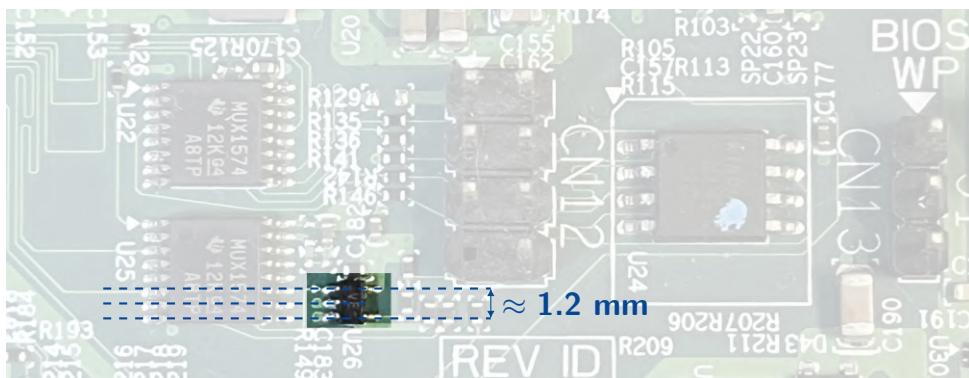
A PROCEDIMENTOS PARA IDENTIFICAÇÃO - U26

O capítulo apresenta os procedimentos adotados para identificar o componente U26, acerca do qual não havia informações disponíveis. Para essa finalidade, o componente foi removido do circuito original e teve sua lógica analisada, conforme descrito nas seções seguintes.

A.1 IDENTIFICAÇÃO PARCIAL

Por se tratar de um CI de dimensões reduzidas, sua identificação direta não é viável. Entretanto, é possível restringir as alternativas considerando o tipo de encapsulamento, a marcação presente, o número de pinos e o espaçamento entre eles. Essa distância pode ser estimada tomando como referência o componente U25, localizado ao lado do dispositivo analisado.

Figura 46 – Distância estimada entre os pinos



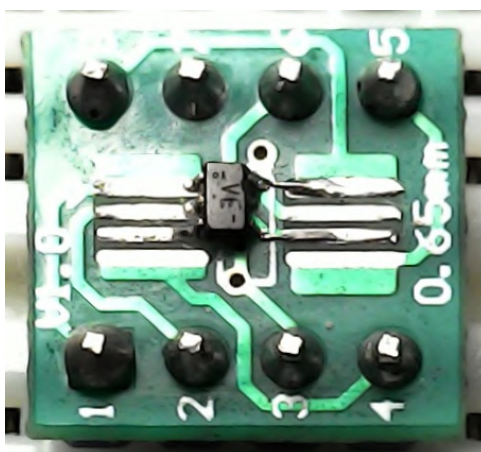
Fonte: Autor

A Figura 46 destaca o componente, que possui cinco pinos e apresenta a marcação “VE” em seu encapsulamento. A figura também evidencia que o espaçamento entre os *pads* do CI é de aproximadamente 0.6 mm. Com base nessas características, pode-se inferir que o encapsulamento pertence à família SOT-25, SOT-353 ou TSSOP-5. Uma consulta ao [S-Manuals \(2025\)](#) indica que o componente corresponde a uma porta AND ou a um *switch* analógico.

A.2 REALIZAÇÃO DE TESTES

A Figura 47 mostra o componente soldado em um adaptador SMD para que os testes pudessem ser feitos em um *protoboard*.

Figura 47 – Componente U26 soldado em adaptador SMD visto por meio de microscópio

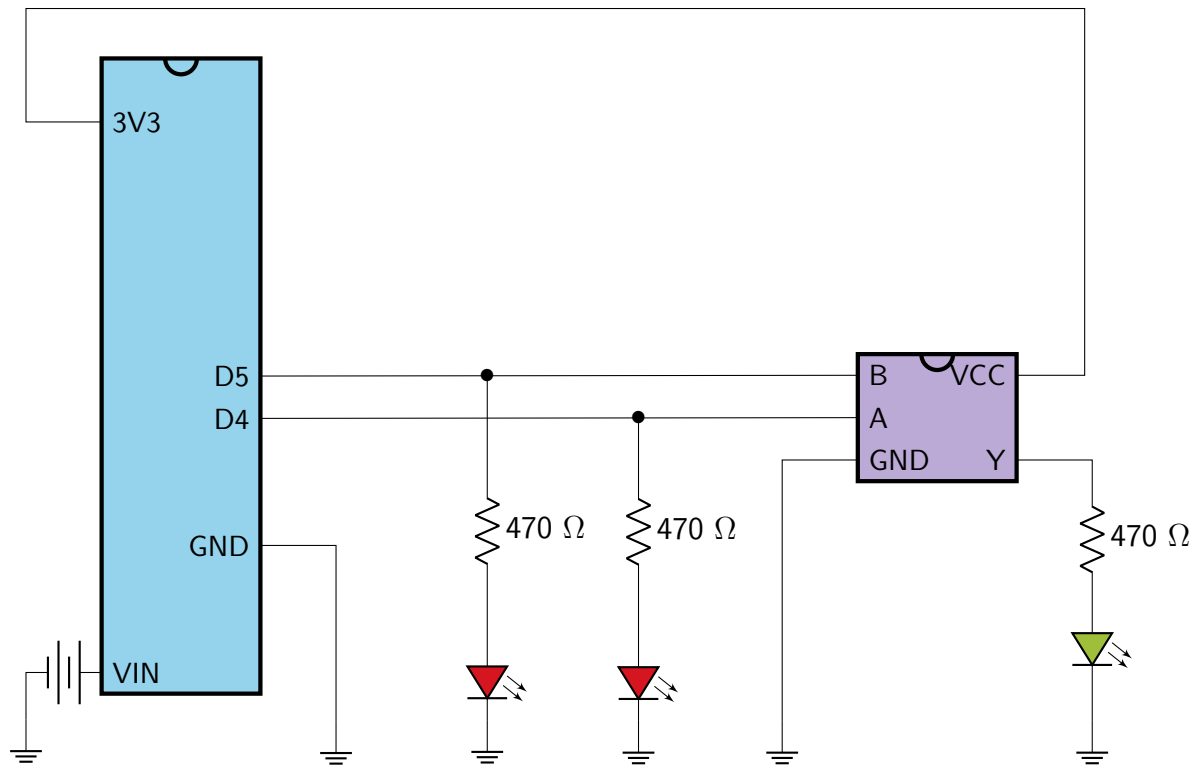


Fonte: Autor

Embora o encapsulamento não coincidissem com o do adaptador, o espaçamento entre os pinos era suficiente para a realização dos testes. Além disso, foi necessário prolongar os pinos 4 e 5 utilizando fio esmaltado, de modo a viabilizar a soldagem aos *pads*, uma vez que o comprimento do componente era menor do que o requerido pelo adaptador. A identificação funcional do componente não pode ser realizada apenas pela medição da tensão em um único terminal, uma vez que tanto um *switch* analógico quanto uma porta lógica AND podem apresentar tensões semelhantes dependendo do circuito em que estão inseridos.

A Figura 48 apresenta o circuito utilizado para testar o componente, implementado com um Arduino Nano. O objetivo do arranjo é variar as tensões aplicadas às entradas e observar como essas variações se refletem na saída. Para facilitar a visualização dos estados lógicos, tanto as entradas quanto a saída foram equipadas com LEDs.

Figura 48 – Circuito para Testes do Componente U26



Fonte: Autor

A Figura 49 apresenta o código do firmware embarcado no Arduino Nano. O objetivo do teste é aplicar as quatro combinações possíveis de entrada e observar a resposta correspondente na saída, sendo cada combinação mantida por um intervalo de um segundo.

Figura 49 – Código de testes a ser embarcado no Arduino Nano



```
const int A = 4; // D4
const int B = 5; // D5

void setup () {
  pinMode(A, OUTPUT);
  pinMode(B, OUTPUT);
  digitalWrite(A, LOW);
  digitalWrite(B, LOW);
}

void loop () {

  // Caso 1: A = 0, B = 0
  digitalWrite(A, LOW);
  digitalWrite(B, LOW);
  delay(1000);

  // Caso 2: A = 0, B = 1
  digitalWrite(A, LOW);
  digitalWrite(B, HIGH);
  delay(1000);

  // Caso 3: A = 1, B = 0
  digitalWrite(A, HIGH);
  digitalWrite(B, LOW);
  delay(1000);

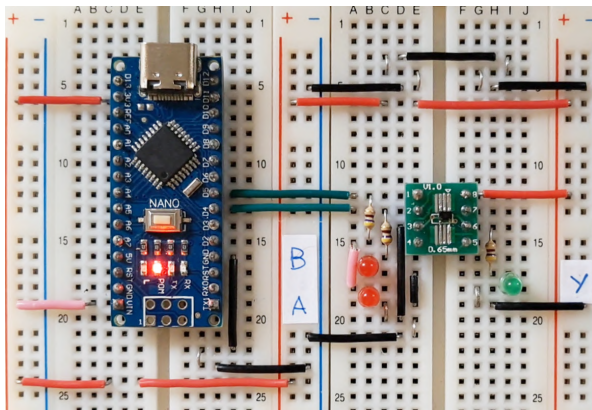
  // Caso 4: A = 1, B = 1
  digitalWrite(A, HIGH);
  digitalWrite(B, HIGH);
  delay(1000);
}
```

Fonte: Autor

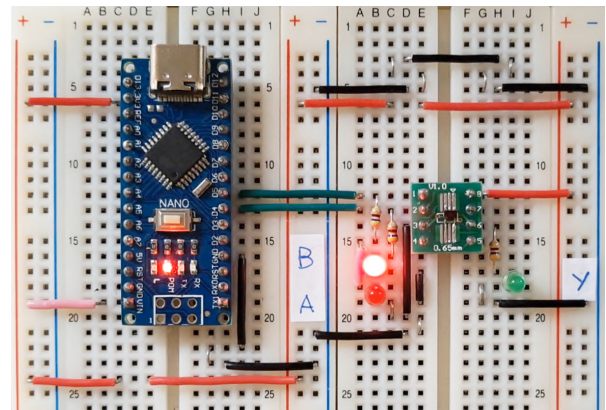
A.3 RESULTADOS

A Figura 50 mostra as quatro configurações possíveis. Como o LED da saída apenas acende quando $A = B = 1$, pode-se concluir que o circuito integrado em questão é uma porta AND de duas entradas.

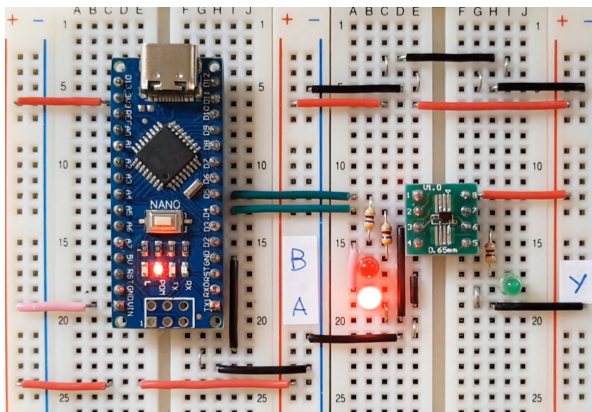
Figura 50 – Comportamento do circuito para diferentes configurações de entrada



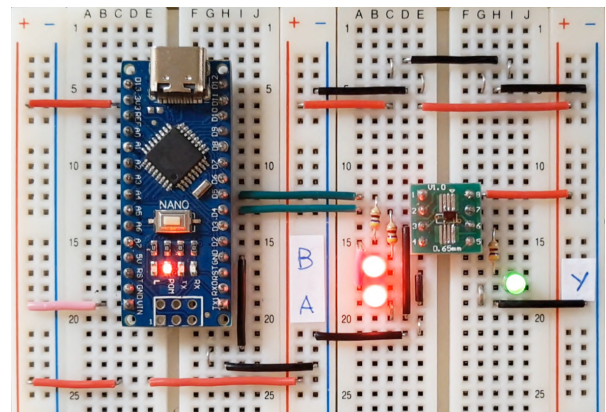
(a) Entradas $A = 0$ e $B = 0$



(b) Entradas $A = 0$ e $B = 1$



(c) Entradas $A = 1$ e $B = 0$



(d) Entradas $A = 1$ e $B = 1$

Fonte: Autor

Diante do comportamento observado do circuito integrado, é possível inferir que o componente corresponde a uma porta AND. Essa conclusão decorre do fato de que, caso se tratasse de um *switch* analógico, o terminal correspondente a Y não apresentaria comportamento compatível com o de uma saída lógica.